

# Deep learning models security: A systematic review

Twinkle Tyagi, Amit Kumar Singh<sup>\*</sup>

Department of CSE, NIT Patna, Patna, Bihar, India

## ARTICLE INFO

### Keywords:

Deep learning  
Security  
Privacy  
Watermarking  
Encryption  
Fingerprint

## ABSTRACT

Deep learning models and the digital records they generate have remarkably increased their adoption of many practical applications. While the success of deep learning in multimedia applications, especially images, helps tackle some of the most challenging problems, one of its copyright violations, ownership conflict, poses a grave concern for many potential applications. Many works on intellectual property protection for such models have proposed to verify ownership. Therefore, it is necessary to conduct a comprehensive study on the security of deep learning models to evaluate their strong background, state-of-the-art solutions, possible attacks, current limitations and notable improvements. This survey attempts to systematically discuss and summarise the recent advanced security solutions for deep learning models through watermarking, encryption and fingerprinting. Our study explores the recent applications, possible attacks, current limitations and notable suggestions regarding deep learning. It also comprehensively evaluates the recent research gaps and opportunities in detail to empower researchers and practitioners to provide additional secure solutions for deep learning models. This extensive survey is the first to consider model security through several notable techniques.

## 1. Introduction

Artificial intelligence (AI) systems impact many lives, both positively and negatively [1,2]. Presently, AI-based systems and its branches, such as machine learning (ML)/deep learning (DL), refer to computer systems capable of executing complex tasks, including reasoning, decision-making and problem-solving [3,4]. Recent promising applications of these models are depicted in Fig. 1. Due to growing interest in various applications [5–14] in recent days, the security of these models is a critical concern for the research communities [15]. Additionally, it is reported that DL models are very costly (refer to Table 1) and require skilled professionals to manage them. As a result, the copyright protection of DL models has become urgent. Further, recent statistics indicate that DL models have exhibited encouraging results for image-processing tasks [16,17]. According to a 2023 report on AI image statistics [18,19], users generate more than 34 million images per day with the help of the DALL-E 2 system developed by OpenAI. Due to the high value of these images in several applications, there is a pressing need to protect their copyright and prevent privacy leakage [20,21]. Therefore, copyright protection of deep learning models has become absolutely necessary.

Watermarking techniques are regarded as a significant line of defence for the copyright protection of DL models [24] and image information [25,26]. The technique embeds confidential data, called a watermark, in an undetectable manner [25]. When the copyright of watermarked files/models is infringed, these hidden marks can be extracted to verify the ownership [24–27]. Thus, it can efficiently reduce the attention of eavesdroppers and avoid their damage. To be effective, the watermarking algorithm must satisfy the

<sup>\*</sup> Corresponding author.

E-mail addresses: [twinklet.ph23.cs@nitp.ac.in](mailto:twinklet.ph23.cs@nitp.ac.in) (T. Tyagi), [amit.singh@nitp.ac.in](mailto:amit.singh@nitp.ac.in) (A.K. Singh).

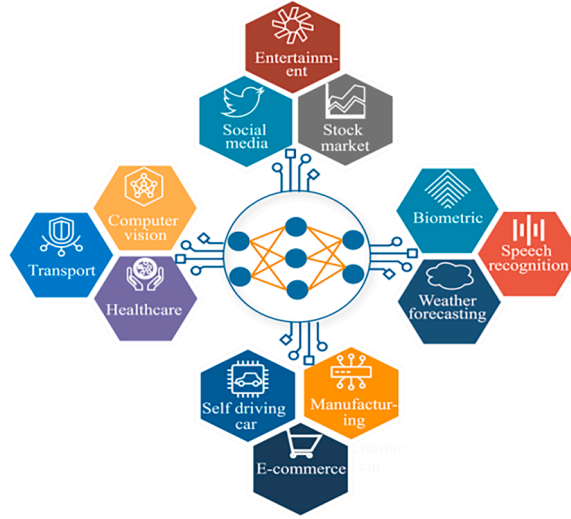


Fig. 1. Application of deep learning.

Table 1

Training cost and training time and computing cost of some well-known models.

| Notable parameters       | BERT [22] | Grover [22] | XLNet [22] | AlphaGo [22]  | ChatGPT3 [23]             | ChatGPT4 [23]           |
|--------------------------|-----------|-------------|------------|---------------|---------------------------|-------------------------|
| Training time            | 4 days    | 2 weeks     | 2.5 days   | 4–6 months    | 6 h                       | 90–100 days             |
| Training cost            | \$7000    | \$25,000    | \$60,000   | \$35 millions | \$2 million - \$4 million | More than \$100 million |
| Computing infrastructure | High      | High        | Medium     | High          | High                      | High                    |

following requirements [28,29]:

- Fidelity—The task accuracy of the deep neural network (DNN) model should not degrade after watermark embedding.
- Robustness—Embedded watermarks should be detectable with high accuracy even after watermark removal attempts.
- Capacity—Ability to embed large amounts of watermarking data.

Fidelity, robustness and capacity are the most crucial requirements for DNN watermarking algorithms. However, these requirements are trade-offs to each other. Security and cost efficiency are equally essential considerations for the watermarking algorithm. In the case of security, the watermark should be secret and not accessible, readable or modifiable by unauthorised users. Efficiency, i.e., time cost, pertains to the time taken to embed and extract the watermark information. Fig. 2 shows the five notable requirements of DNN watermarking algorithms.

### 1.1. Contribution and related surveys

Several survey papers have been published raising concerns over the intellectual property (IP) protection of DL models; however, to the best of our knowledge, no paper has a comprehensive review of model security based on watermarking, encryption and fingerprinting simultaneously. Further, this survey is the first to summarise and compare the watermarking, fingerprinting and encryption techniques used for model security. It attempts to systematically discuss and summarise the recent state-of-the-art copyright protection and other security solutions for DL models. Specifically, we aim to investigate the following research questions:

- RQ-1.** What kind of attacks affects the performance of DL models? And how do these attacks impact them?
- RQ-2.** What are the state-of-the-art solutions to the security issues regarding DL models?
- RQ-3.** What are the notable techniques being employed, and what are their objectives, datasets, models, results and limitations?
- RQ-4.** How are the performances of these models evaluated?
- RQ-5.** What are the recent challenges to DL models security and possible solutions?

Although DL models have a significant and growing [16] role in various applications, relatively few surveys are available regarding their copyright protection.

The author overviews the most recent DNN watermarking techniques for the IP protection of DNN models in Li et al. [20]. The paper discusses various DNN requirements, including robustness, fidelity and security. In [30], the author discusses the watermarking solutions for the DNNs that can protect the owner's IP and verify the model's ownership. The survey starts with various attacks on

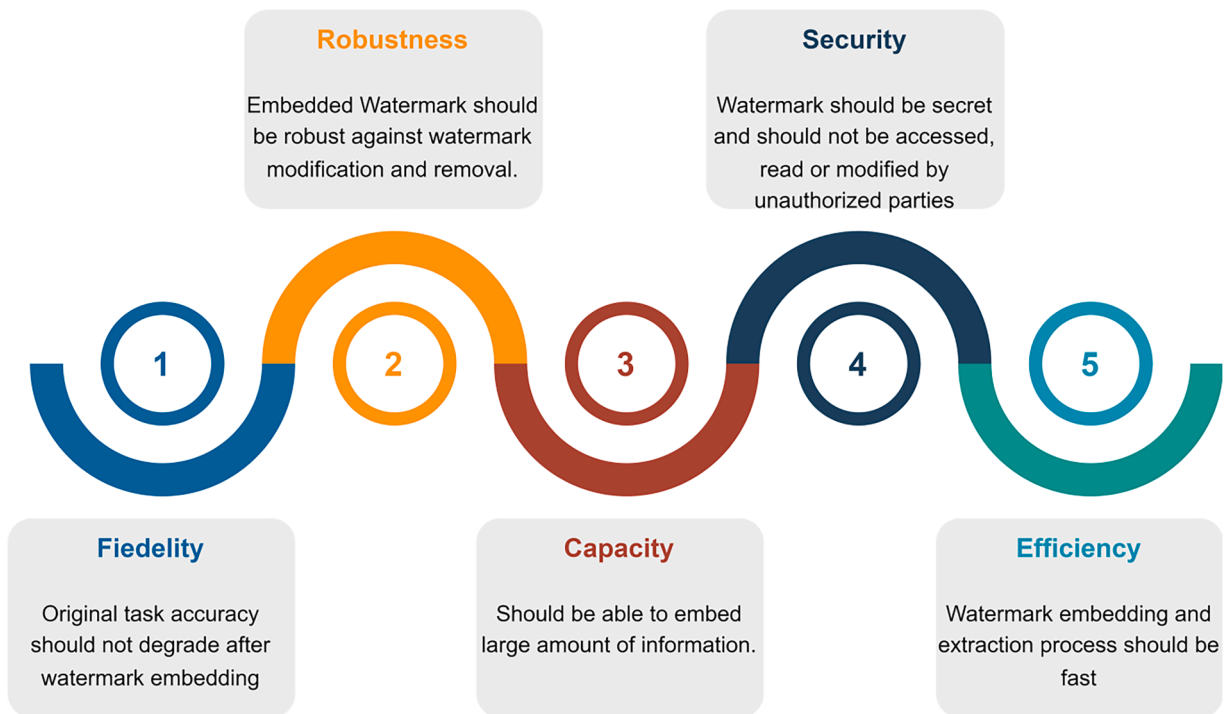


Fig. 2. Requirements for effective watermarking algorithm.

model security and discusses threat models and security goals regarding DNN watermarking. Further, DNN watermarking methods are categorised based on the user's access to the model. The survey concludes with a discussion about potential shortcomings, watermarking requirements and limitations. However, it does not provide any possible solution to the limitations presented. In [31], the author summarises watermarking and fingerprinting techniques for DNNs' IP protection. Threat models for fingerprinting and watermarking are discussed. The watermarking techniques are classified as white-box and black-box. Literature on a few watermarking and fingerprinting techniques is discussed, along with attacks on them. However, the survey does not provide an overview of potential attacks on the model. In [32], the author presents a comparative study of DNN models for copyright protection. Watermarking techniques for DNN model security and multimedia security are covered. The trade-off between imperceptibility, robustness, capacity and security is established. Limited literature for both white-box and black-box watermarking is discussed. In [33], the author presents a detailed literature review for deep IP protection. The review starts with a background problem discussion and discusses performance evaluation criteria and threats to deep IPs. Various threats to model security and watermarking are discussed. Watermarking and fingerprinting approaches are discussed in detail. The survey also includes future directions and possible solutions to IP theft. In [34], the author mainly focuses on the various attacks on the trigger set-based watermarking approach. The survey summarises trigger set-based watermarking to evaluate its robustness and then discusses attacks on it. The defence of different watermarking techniques against attacks is evaluated. In [35], the author briefly reviews various techniques that can be used to protect the IP of DL models. A taxonomy of DNN IP protection methods is proposed. Watermarking and fingerprinting schemes are considered and summarised. Attacks on the model security and watermarking and fingerprinting scheme are discussed. The survey ends with a discussion on challenges to protecting DNN IP and its future scope. In [36], the author mainly focuses on a review of preserving the privacy of DL models for healthcare applications. The survey starts by discussing various DL models used to manage and process medical data. The author discusses the threat model and various attacks on the privacy of the DL model, categorises them and presents possible solutions. Further, the survey discusses privacy-preserving computation in DL networks used in healthcare data processing and future research directions. In [37], the author presents a detailed review of privacy and security issues in DL models. It covers an overview of attacks and defences on privacy, with various attacks discussed in detail along with possible defences. This survey also compares various defence methods based on their advantages and shortcomings. The survey [38] primarily briefly introduces privacy-preserving tools and then explores threat models, examining the various threats. Data encryption-based privacy-preserving techniques are categorised based on linear and non-linear computation. Different ways to defend against the attacks are covered, alongside a focus on technical obstacles. In [39], the survey evaluates the robustness of DNN watermarking techniques. A taxonomy for DNN watermarking schemes and attacks to remove watermarks is proposed in this survey. The survey evaluates the robustness of various DNN watermarking schemes against attacks and presents a watermark-robustness toolbox to assess the watermark's robustness. The author demonstrates that combining different attack methods can effectively remove the embedded watermarks using all the DNN watermarking techniques examined in the survey. In [40], the author presents an overview of the DNNs and possible defences. Deep neural network IP protection techniques are classified according to their protection methodologies and discussed in detail along

with various attacks. The DNN IP protection techniques discussed and compared based on the evaluation metrics.

This survey attempts to systematically discuss and summarise the recent state-of-the-art security solutions for DL models. Our study explores recent applications, possible attacks, limitations, and notable suggestions. Further, this comprehensive survey explores the recent research gaps and opportunities in detail; through this, we hope to empower researchers and practitioners to provide increasingly secure solutions for DL models. Moreover, the extensive survey is the first to simultaneously consider model security through watermarking, encryption and fingerprinting. Table 2 represents a comparative analysis of our survey and existing state-of-the-art survey papers in this area.

### 1.2. Attacks scenario on deep learning models

In recent years, DL models have made significant advances in performance due to their powerful learning and feature expression capabilities and become popular in many potential applications [5-14]. However, these models may be susceptible to attacks involving serious privacy and security issues [41]. Fig. 3 presents the possible attack scenarios on DL models. The model owners train the model on data using their own resources; however, attackers can steal the model by implementing model extraction attacks [42] and using it for their financial gains. In a model extraction attack, the attackers duplicate the model's parameters/hyper-parameters and use them for their benefit. Further, adversaries can manipulate training data or insert backdoors, resulting in targeted misclassification. Adversaries can also try to infer personal sensitive information from the model, which is called a model inversion attack [43]. Both model extraction and model inversion attacks are threats to model privacy [44]. Differential privacy [45], homomorphic encryption [46], secure multi-party computation [47], and a trusted execution environment [48] are four popular mainstream technologies that preserve privacy in these models. Watermarking techniques offer a valuable solution for preventing copyright violations of DL models [24]. If the copyright of watermarked files/models is infringed, the watermark can be extracted to solve the ownership conflict, if any. Thus, it can efficiently reduce the attention of eavesdroppers and prevent their potential damage. Fig. 4 highlights the general copyright protection framework of DL models using a watermarking scheme. Here, model owner embeds small user information, called a watermark, into the DNN model in the watermark embedding phase to get watermarked model. At receiver side, the user will extract the hidden mark (s) to verify the ownership of the modal. A watermark can be embedded into the DNN model during training or fine-tuning the model. Watermarking can be reversible [49] for DNN models, where the original model can be restored after the extraction process or non-reversible [24], where the watermark remains within the model even after the extraction process. Furthermore, watermarking in DNN models can be classified as white-box or black-box based on the extraction procedure. If the internal parameters are accessible to the watermark verifier, we may conclude that the watermark has been recovered through the white-box method [24].

The watermark can be extracted directly from internal parameters by enquiring about weights or activation functions. If only the final output of the DNN model is available, we may deduce that the watermark was recovered in a black-box manner. The verifier does not have access to the internal structure of the DNN model. The watermark is extracted by querying the model with a carefully designed input called a trigger set [9]. Major threats to DL-based watermarking include fine-tuning, parameter pruning and watermark overwriting attacks [24]. Other than that, the methods to secure the DNN models are susceptible to the following attacks [30].

- a. Forging/Ambiguity attacks: The attacker might be able to forge his watermark or fingerprint and embed it into the original model [50]. The owner's watermark remains in the model and can still be verifiable. However, multiple watermarks or fingerprints create ambiguity, making it difficult to identify the original owner. If the watermark or fingerprint in question is not mapped to its original owner's identity, the attacker can simply claim its ownership [51]. The attacker can also find a fake watermark or fingerprint within the model and claim it as the actual one [52].
- b. Watermark removal attacks [53]: To showcase the stolen model as their own, the attackers try to entirely remove the watermark or fingerprint. To succeed in their attempts, the attackers should have the minimum knowledge about the location of the watermark or fingerprint and the algorithm used for watermarking or fingerprinting. The most common methods used by attackers to remove watermarks and fingerprints are:
  - Fine-tuning [53]: The malicious user can utilise a dataset to fine-tune the previously trained model to modify the model parameters and remove embedded watermarks or fingerprints, as shown in Fig. 5. The attacker can fine-tune just the last layer or all the layers of the model to diminish the user's chances of extracting a verifiable watermark or fingerprint.
  - Pruning [54]: Pruning is an optimisation technique that improves the model's efficiency by selecting and removing redundant parameters without affecting the model's accuracy. However, malicious users can use this technique to effectively remove embedded watermarks or fingerprints from the model, as shown in Fig. 6. The malicious users can prune the model by eliminating the weight parameters of the model, the neurons from the underlying layer or entire filters in the convolution layers.
  - Model compression [55]: Model compression reduces the model size without significantly affecting its task accuracy, making it more efficient. However, malicious users can use model compression to remove watermarks or fingerprints. Model compression is performed by pruning the neurons or removing parameters of very low significance. Removing parameters and neurons can affect the detectability of the watermark or fingerprint.
  - Transfer learning [27]: Transfer learning is a technique where a model trained for one task is re-used for another related task. The trained model can be used to train another model with small changes to its layers and parameters. This saves considerable time and computational power, but the minor changes within model layers may lead to watermark or fingerprint removal.

**Table 2**

Comparison of our survey with existing state-of-the-art survey papers.

| Ref No.         | Model security solutions via |                |            | Discussed attacks on |           |             |            | Role of watermarking, fingerprinting and encryption for model security | Attack scenario and related performance metric | Comparison of state-of-the-art security solutions | Challenges and opportunities |
|-----------------|------------------------------|----------------|------------|----------------------|-----------|-------------|------------|------------------------------------------------------------------------|------------------------------------------------|---------------------------------------------------|------------------------------|
|                 | Watermarking                 | Fingerprinting | Encryption | Model                | Watermark | Fingerprint | Encryption |                                                                        |                                                |                                                   |                              |
| [20]            | ✓                            | ×              | ×          | ×                    | ×         | ×           | ×          | ✓                                                                      | ×                                              | ×                                                 | ×                            |
| [30]            | ✓                            | ×              | ×          | ✓                    | ✓         | ×           | ×          | ✓                                                                      | ×                                              | ×                                                 | ✓                            |
| [31]            | ✓                            | ×              | ×          | ×                    | ✓         | ✓           | ×          | ✓                                                                      | ×                                              | ×                                                 | ×                            |
| [32]            | ✓                            | ×              | ×          | ×                    | ×         | ×           | ×          | ×                                                                      | ×                                              | ×                                                 | ✓                            |
| [33]            | ✓                            | ✓              | ×          | ✓                    | ✓         | ✓           | ×          | ✓                                                                      | ×                                              | ×                                                 | ✓                            |
| [34]            | ✓                            | ×              | ×          | ✓                    | ✓         | ×           | ×          | ×                                                                      | ×                                              | ×                                                 | ×                            |
| [35]            | ✓                            | ✓              | ×          | ✓                    | ✓         | ✓           | ×          | ✓                                                                      | ×                                              | ×                                                 | ✓                            |
| [36]            | ×                            | ×              | ✓          | ✓                    | ×         | ×           | ×          | ×                                                                      | ×                                              | ×                                                 | ✓                            |
| [37]            | ×                            | ×              | ✓          | ✓                    | ×         | ×           | ✓          | ✓                                                                      | ×                                              | ×                                                 | ✓                            |
| [38]            | ×                            | ×              | ✓          | ✓                    | ×         | ×           | ✓          | ✓                                                                      | ×                                              | ×                                                 | ✓                            |
| [39]            | ✓                            | ×              | ×          | ×                    | ✓         | ×           | ×          | ✓                                                                      | ×                                              | ×                                                 | ×                            |
| [40]            | ✓                            | ✓              | ✓          | ×                    | ✓         | ✓           | ✓          | ×                                                                      | ×                                              | ✓                                                 | ✓                            |
| <b>Our Work</b> | ✓                            | ✓              | ✓          | ✓                    | ✓         | ✓           | ✓          | ✓                                                                      | ✓                                              | ✓                                                 | ✓                            |

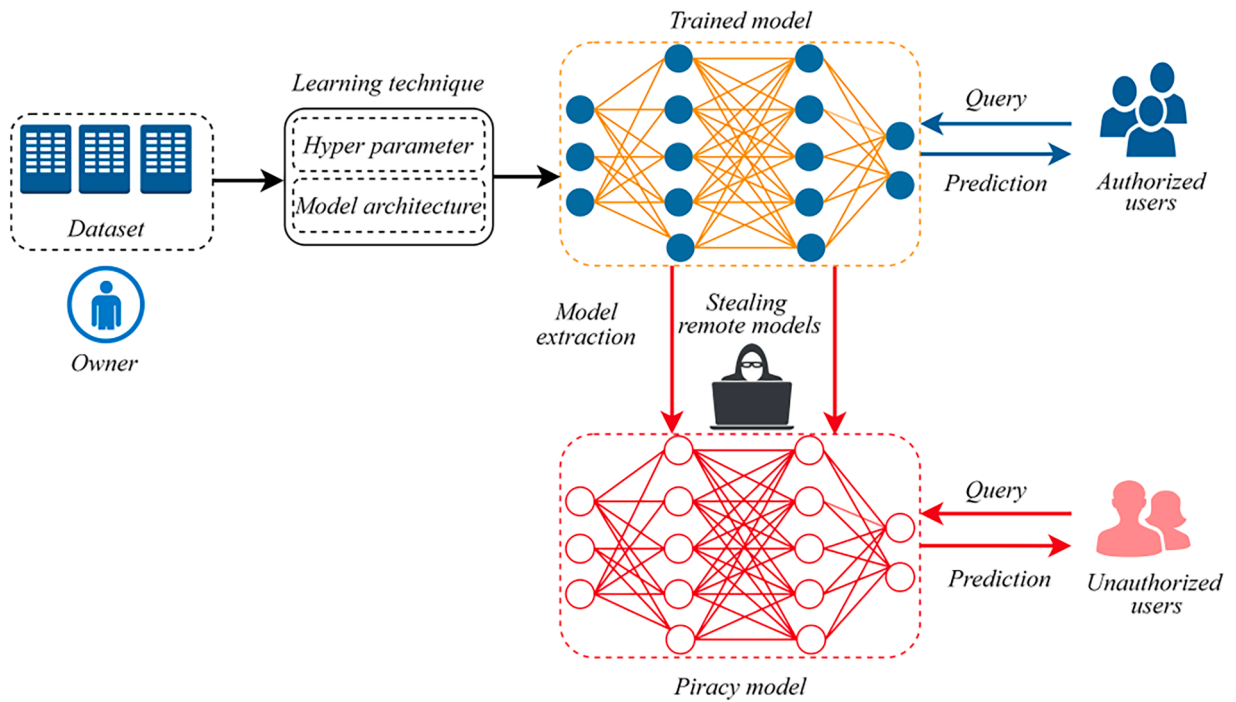


Fig. 3. Deep learning model stealing and intellectual property infringement.

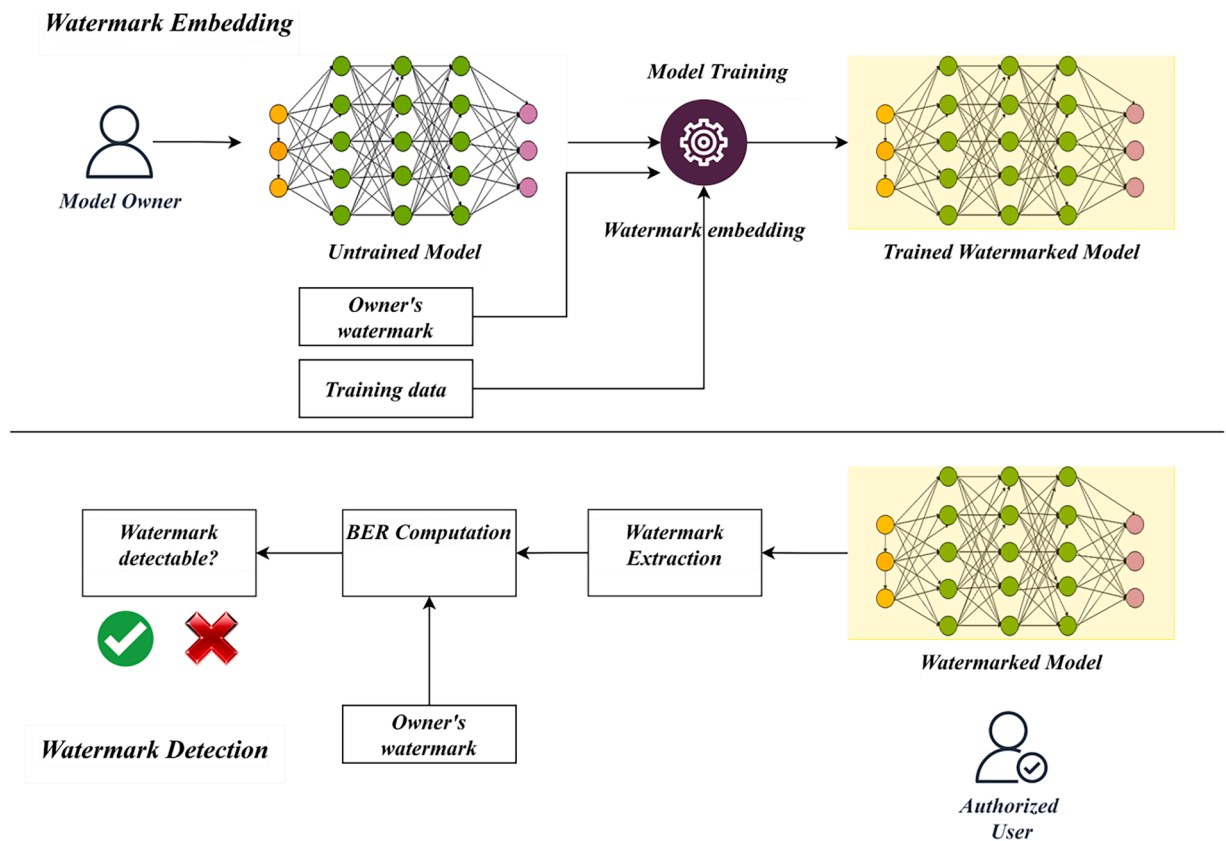


Fig. 4. Copyright protection solution of deep learning model through watermarking.



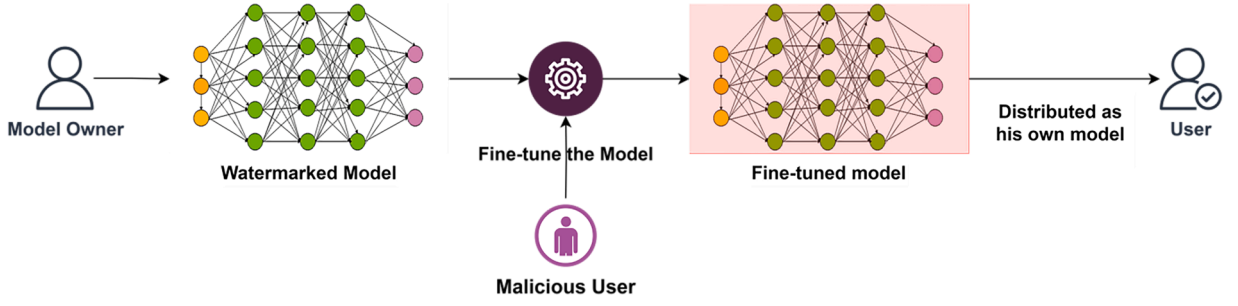


Fig. 5. Fine-Tuning attack on embedded watermark.

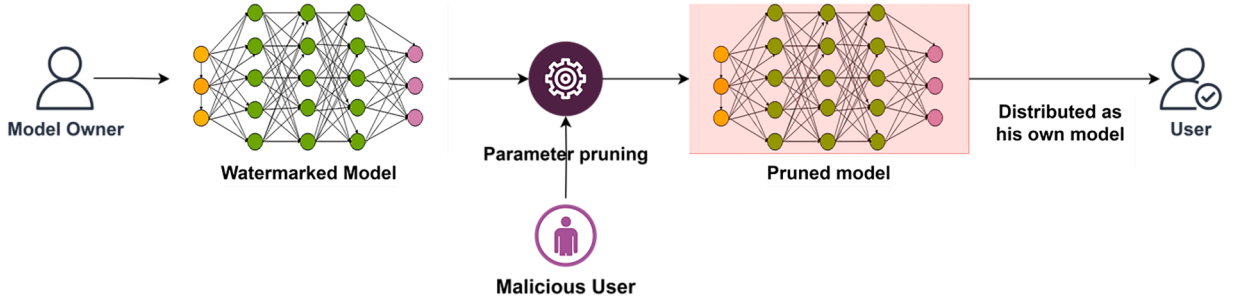


Fig. 6. Parameter pruning attack on embedded watermark.

- **Distillation [56]:** Distillation is the process of creating a smaller, simpler model from a larger, more complex one by transferring knowledge. While transferring knowledge from the larger model to a smaller one, it is not guaranteed that the watermark or fingerprint will also be transmitted.
- **Backdoor removal [57]:** Backdoor removal works by eliminating the functionalities that are not relevant to the original task. If the watermark is embedded in the functionality of the source model as a backdoor, then it could also be removed along with the backdoor.
- c. **Watermark overwriting attacks [58]:** Malicious users can also attempt to overwrite existing embedded watermarks with their own watermarks, as shown in Fig. 7. To achieve this, they deploy their watermarking data to fine-tune the watermarked model, causing their watermark to be embedded over the existing watermark of the original user. This overwrites the existing watermark, making it challenging to verify the ownership of the watermarked model.
- d. **Adaptive attacks:** Attackers attempt to remove, alter or forge the watermark or fingerprint embedded into the DNN model by continuously adjusting their strategy based on the knowledge of the watermarking scheme.
- e. **Hybrid attacks:** The attacker can use combination of multiple attacks, i.e., it may be combination of fine-tuning and pruning attacks, at a time to remove our hidden mark.
- f. **Collusion attacks:** In a collusion attack, multiple users possessing their own copy of the watermarked or fingerprinted DNN model collaborate to identify the location of the watermark and ultimately remove it. Attackers utilise the differences between their models to locate and remove the watermark.

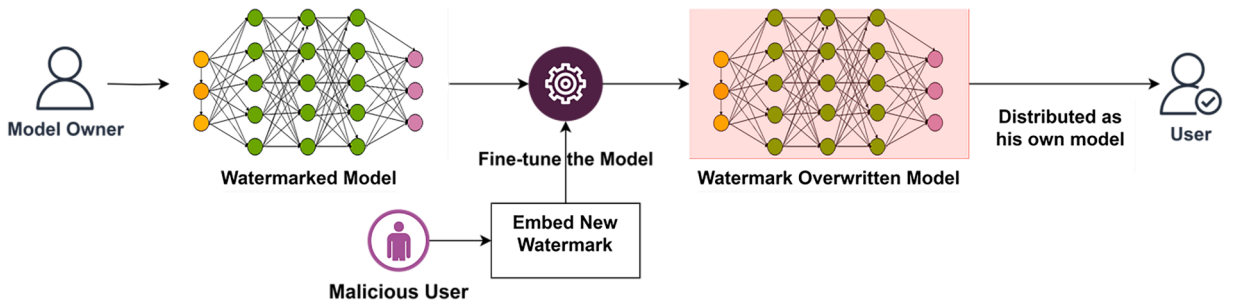


Fig. 7. Watermark overwriting attack on embedded watermark.

### 1.3. Performance metric

This section describes popular metrics and measures (Refer to Table 3) such as bit error rate (BER) [59], model accuracy degradation, attack success rate, mean squared error (MSE) [60], false positive and negative rate, precision and recall, F1 Score, and watermark removal rate for estimating the performance of deep learning model against attacks. We have summarized the important metric to measure the performance of deep learning model against attacks in Table 3. Further, key performance indicator for model security using watermarking is depicted in Fig. 8. The few notable metric are listed as follows:

- Bit error rate (BER)—BER evaluates the difference between the model owner's watermark and extracted watermark. BER is used to determine if the extracted watermark matches the model owner's watermark or not.
- Model accuracy degradation—Model accuracy degradation is used to measure the reduction in accuracy following an attack. Using Model accuracy degradation, we can identify whether the model is attacked by adversary or not.
- Attack success rate—Attack success rate evaluates the number of times the adversary was able to successfully attack the model.
- Mean squared error (MSE)—MSE is the loss function that measures the loss of accuracy after the attack.
- False positive rate—False positive rate evaluates the rate at which non-watermarked models are identified as watermarked model.
- False negative rate—False negative rate evaluates the rate at which watermarked models are identified as non-watermarked model.
- Precision and recall—Precision and recall provides detailed information about the attack's influence on robustness and watermark detection capabilities. High precision guarantees that watermark detection mechanism is accurate, while high recall ensures that watermark detection algorithm identifies all watermarked models.
- F1 Score—F1 score is the harmonic mean of precision and recall. The F1 score helps assess how effectively the watermark detection system works when under attack.
- Watermark removal rate—Watermark removal rate refers to the rate at which the watermark may be effectively removed from the model.

The rest of the article is organised as follows: Section 2 presents a detailed survey of model security techniques, followed by recent challenges discussed in Section 3, and finally, the conclusion is presented in Section 4.

## 2. Model security techniques

With the rapid development of AI and beyond in recent years, DL has been widely used in various research areas. However, these models often struggle in challenging scenarios, such as copyright violation or ownership conflicts due to theft and attacks. Watermarking and fingerprinting technologies include effective strategies to address the challenges of model security, enhancing the copyright protection of the model and preventing ownership conflicts, if any. Additionally, strong encryption techniques also provide high levels of safety and privacy. Privacy protection is aimed at individuals who wish to prevent models/data that are considered privacy-sensitive from entering the public domain. The following sub-sections, 2.1, 2.2 and 2.3, present highly advanced security and privacy solutions for DL models using watermarking, fingerprinting and encryption studies.

### 2.1. Model protection through watermarking

Watermarking is a useful strategy to protect the IP of DNNs. Watermarking is the process of embedding small user information, called a watermark, into the DNN model so that IP thefts can be easily detected and the ownership of the DNN model verified. According to available literature, DL protection through watermarking is divided into two categories: white-box and black-box watermarking.

#### 2.1.1. White-box watermarking

In white-box watermarking, the owner embeds personal information, called the watermark, into the model weights or activations [24]. The initial processes of white-box watermark embedding and verification are depicted in Fig. 9. Watermark is either embedded into the model parameters at various locations at the time of model training, as compiled in Table 4, or on the trained model by fine-tuning the model with the watermarking dataset. The user has complete access to the model and can extract the model's weights and other parameters. The user extracts the watermark from the suspect model parameters and compares it with the owner's watermark. If the bit error rate (BER) is lower than a certain threshold, the watermark is the same as the owner's, and the suspect model is a replica of the original model. In a 2017 study [24], the author introduced the concept of DNN watermarking to protect the IP and detect IP infringement. The author embedded the watermark in one of the layers of deep convolutional neural networks (DCNN) while training the model and added a parameter regulariser to the original cost function. The scheme was evaluated using CIFAR10 and Caltech101 datasets on a wide residual network. Results showed that the test error rate (TER) for the embedded and un-embedded networks is the same, while the capacity of the embedded watermark is limited by the number of parameters of the layer in which the watermark is embedded. The scheme is robust against fine-tuning attacks but weak against watermark overwriting and network morphism attacks. In [61], the author proposed a white-box watermarking technique based on embedding watermarks using a spread spectrum paradigm to protect the IP rights associated with DNN models. A watermark was embedded by adding a new regularisation term into the loss function. The method was evaluated on ResNet and VGG16 models using three standard datasets. The Test results showed satisfactory performance with significantly low TER and BER. The TER and BER remain almost the same after fine-tuning and



Table 3

Performance metric against considered attacks.

| Used metric                | Description                                                                                                                          | Formula                                                                                                                                                  | Attacks considered |                 |                  |                 |                    |                |              |                   |                       |           |         |                 |                |                  |                                         | Key points |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-----------------|------------------|-----------------|--------------------|----------------|--------------|-------------------|-----------------------|-----------|---------|-----------------|----------------|------------------|-----------------------------------------|------------|
|                            |                                                                                                                                      |                                                                                                                                                          | Adversarial        | Model Inversion | Model Extraction | Model Poisoning | Trojan or Backdoor | Gradient-based | Side-channel | Watermark removal | Watermark overwriting | Collusion | Privacy | Removal attacks | Forged attacks | Ambiguity attack |                                         |            |
| Bit error rate             | Bit error rate (BER) is the number of bit errors in owner's watermark bit and extracted watermark bit divided by total watermark bit | $BER = B_e/B_n$ , where $B_e$ is bit difference in owner's watermark and extracted watermark, $B_n$ is total no of watermark bits                        | x                  | x               | ✓                | x               | x                  | x              | x            | ✓                 | ✓                     | ✓         | x       | x               | x              | x                | BER should be 0.0                       |            |
| Model accuracy degradation | Measures the drop in accuracy after an attack                                                                                        | $\Delta Accuracy = Accuracy_{before attack} - Accuracy_{after attack}$                                                                                   | ✓                  | ✓               | ✓                | ✓               | ✓                  | ✓              | x            | ✓                 | ✓                     | ✓         | x       | ✓               | ✓              | ✓                | Should be very high                     |            |
| Attack success rate        | Measure the time for which attack was successful                                                                                     | $ASR = \frac{Successful\ attacks}{Total\ attacks} \times 100\%$                                                                                          | ✓                  | x               | x                | ✓               | ✓                  | x              | ✓            | x                 | x                     | ✓         | x       | x               | x              | x                | Should be close to 0                    |            |
| Mean squared error (MSE)   | Measures impact on accuracy after the attack                                                                                         | $MSE = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$ where $N$ is no. of observations, $y_i$ is the observed value, $\hat{y}_i$ is corresponding predicted value | x                  | ✓               | x                | x               | x                  | ✓              | x            | x                 | x                     | x         | ✓       | x               | x              | x                | Lower the value indicates better result |            |
| False positive rate        | Measures the rate at which non-watermarked models are identified as watermarked model                                                | $FPR = \frac{F_+}{(F_+ + T_-)}$ Where $F_+$ is False positive, $T_-$ is true negative                                                                    | x                  | x               | x                | ✓               | x                  | x              | x            | ✓                 | ✓                     | ✓         | x       | x               | ✓              | x                | Should be close to 0                    |            |
| False negative rate        | Measures the rate at which watermarked models are identified as non-watermarked model                                                | $FNR = \frac{F_-}{(F_- + T_+)}$ Where $F_-$ is False negative, $T_+$ is true positive                                                                    | x                  | x               | x                | ✓               | x                  | x              | x            | ✓                 | ✓                     | ✓         | x       | x               | ✓              | x                | Should be close to 0                    |            |
| Precision and recall       | Measure the effectiveness of the attacks                                                                                             | $Precision = \frac{T_+}{T_+ + F_+}$<br>$Recall = \frac{T_+}{T_+ + F_-}$ Where $T_+$ is true positive, $F_+$ is False positive, $F_-$ is False negative   | ✓                  | x               | ✓                | x               | x                  | x              | x            | x                 | x                     | x         | ✓       | ✓               | x              | ✓                | Should be more than 80 %                |            |
| F1 Score                   | Measures model's accuracy                                                                                                            | $F1 = \frac{2}{\left(\frac{1}{precision} + \frac{1}{recall}\right)}$                                                                                     | ✓                  | x               | ✓                | x               | x                  | x              | x            | x                 | x                     | ✓         | x       | x               | ✓              | x                | Should be close to 1                    |            |
| Watermark removal rate     | Measures the rate at which watermark can be removed from the model                                                                   | $Removal\ Rate = \frac{Successful\ attempts}{Total\ attempts}$                                                                                           | x                  | x               | ✓                | x               | x                  | x              | x            | ✓                 | ✓                     | ✓         | x       | ✓               | x              | x                | Should be as low as possible            |            |

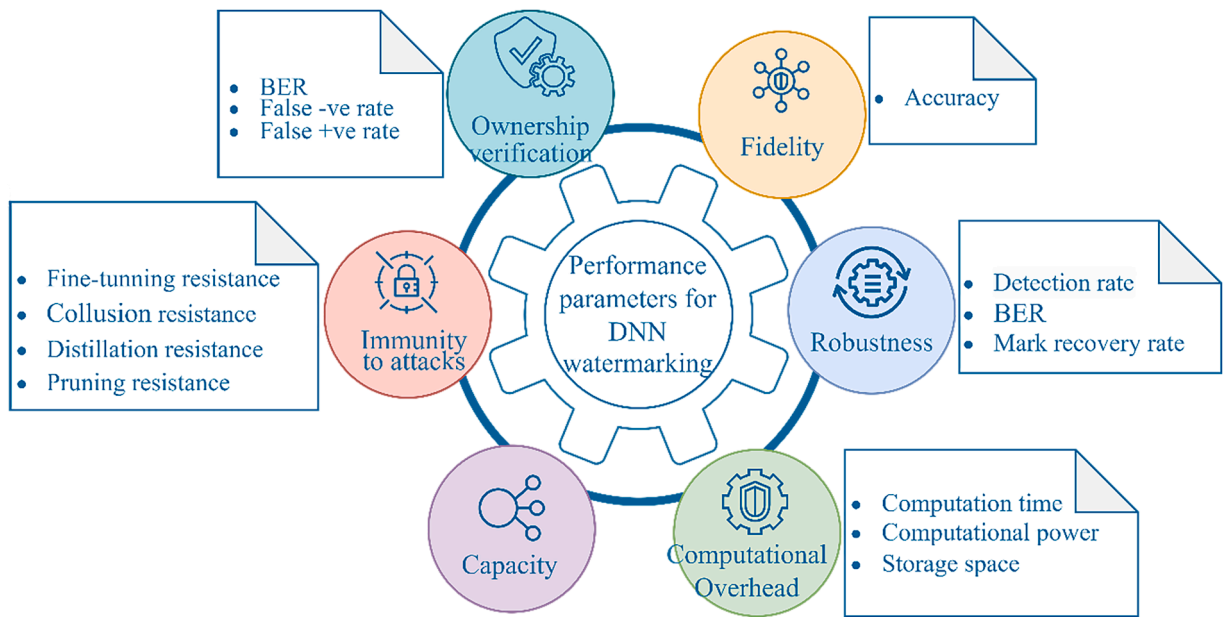
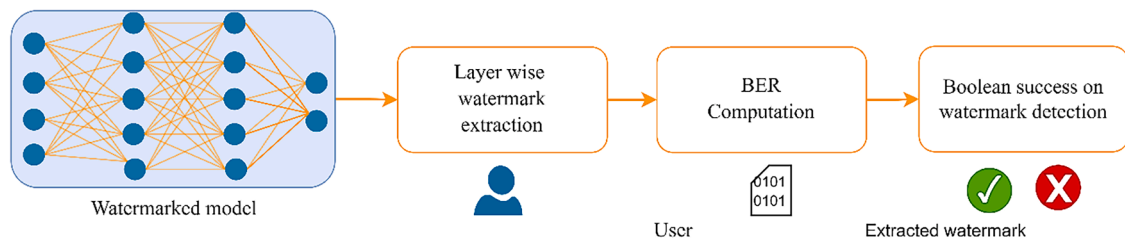
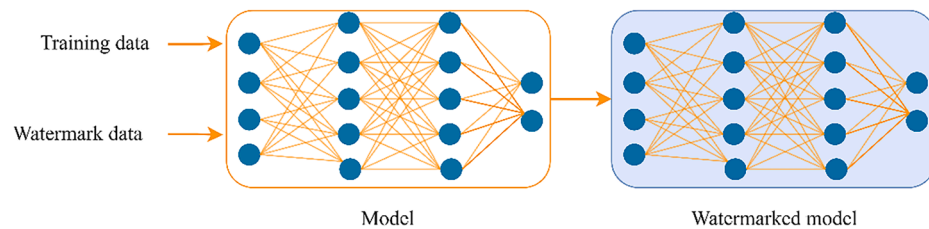


Fig. 8. Key performance indicator for model security using watermarking.

#### Watermark Embedding



#### Watermark Detection

Fig. 9. White-box watermark embedding and verification process.

parameter pruning. However, performance against watermark overwriting attacks has not been verified. The issues regarding protecting the IP rights of the DNN model and watermark overwriting were addressed in Kuribayashi et al. [62]. Watermarks can be embedded in any layer; initially, the watermark was encoded using constant weight code and then embedded into the DNN model. The VGG16, ResNet50 and XceptionNet models trained on the ImageNet dataset were used for evaluation. The accuracy remained practically the same after embedding with minor loss. Non-fungible tokens are used to safeguard against watermark overwriting attacks. In [63], the author addressed verifying the ownership of the model by proposing an interesting watermarking based on the CDMA spread spectrum channel coding. The results showed that model performance does not degrade after embedding the watermark; the

**Table 4**

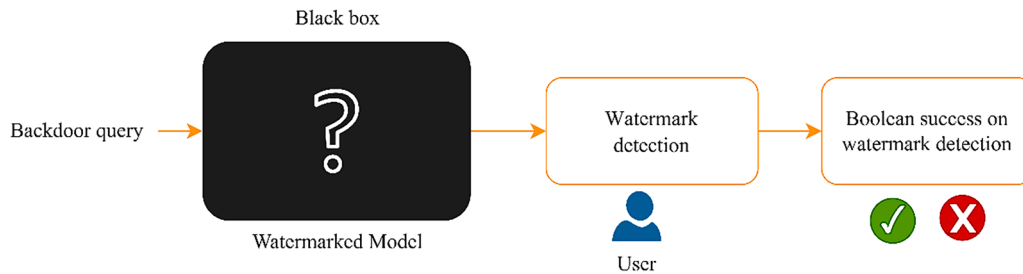
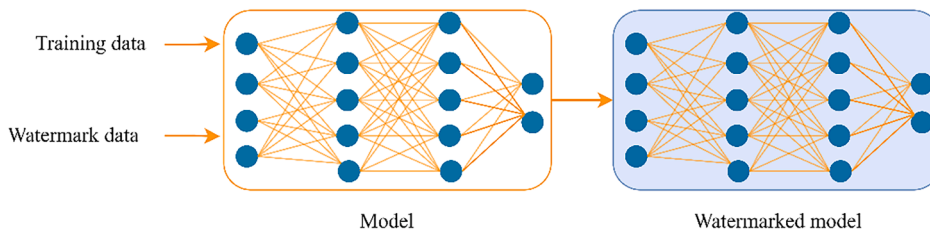
State-of-the-art white-box watermarking techniques.

| RefNo. | Objective                                                               | Technique                                      | Datasets                          | Model                        | Embedding          | Reversible | Performance Metric                                        | Results                                                  | Limitations                                          | Embedding Location                      | Type of watermark                                               | Loss function                           | Attacks                                                                                                              |
|--------|-------------------------------------------------------------------------|------------------------------------------------|-----------------------------------|------------------------------|--------------------|------------|-----------------------------------------------------------|----------------------------------------------------------|------------------------------------------------------|-----------------------------------------|-----------------------------------------------------------------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| [24]   | To protect intellectual property and detect infringement                | New regulariser term is added to loss function | CIFAR-10, Caltech101              | MLP, RNN, LSTM               | Distil-to-embed    | No         | TER, Training Loss, capacity, Robustness                  | Low BER and TER                                          | Weak against watermark overwriting, network morphism | Convolution layer, in parameters        | Bit vector                                                      | Binary cross entropy                    | Fine-tuning or transfer learning, model compression                                                                  |
| [61]   | To verify legitimate owner                                              | Speed Transform Dither Modulation              | CIFAR-10, GTSR, ImageNet          | WRN, ResNet, VGG             | Train-to-embed     | No         | BER, TER, robustness                                      | Low BER and TER                                          | No data for watermark overwriting                    | In weights                              | Bit vector                                                      | Binary cross entropy                    | Fine-tuning, parameter pruning,                                                                                      |
| [62]   | To develop a detector for hidden watermark                              | CWC, NFT(Non-fungible tokens)                  | Flower dataset                    | VGG16, ResNet50, XceptionNet | Train-to-embed     | No         | Accuracy, Loss                                            | High accuracy, small loss                                | –                                                    | Convolution layer, in weight parameters | Bits                                                            | Mean squared error (MSE)                | Transfer learning and fine-tuning, model pruning, watermark overwriting                                              |
| [63]   | To develop a robust Deep neural network watermarking.                   | CDMA spread-spectrum                           | MNIST, CIFAR-10, WikiText, ESC-50 | MLP, VGG-16, RNN, CNN        | Train-to-embed     | No         | Robustness, capacity, secrecy, efficiency, unforgeability | High accuracy, false negative, false negative rate is 0, | –                                                    | Weight parameters                       | Short text watermark of 152 bits, large image watermark of 1 KB | –                                       | Fine-tune all layers, Re-train all layers, Fine-tuning using REFIT framework, parameter pruning, parameter shuffling |
| [64]   | To embed fingerprint within parameters of model                         | PDF, Orthogonal modulation, coded modulation   | MNIST, CIFAR-10                   | CNN, WRN                     | Fine-tune-to-embed | No         | Fidelity, Robustness, fingerprint collusion,              | High accuracy, false rate is 0, robust                   | No data on watermark overwriting attacks             | Convolution layer, in weight parameters | Bits                                                            | Cross-entropy, Mean squared error (MSE) | Model fine-tuning, parameter pruning, collusion attack                                                               |
| [65]   | To develop a watermarking methodology for both white-box and black-box. | Gaussian Mixture Model (GMM)                   | MNIST, CIFAR-10                   | MLP, CNN, WRN (Wide ResNet)  | Fine-tune-to-embed | No         | Fidelity, robustness, reliability                         | High watermark accuracy                                  | No data on watermark overwriting attacks             | Intermediate layers, output layer       | Binary string (Owner's info)                                    | Cross-entropy                           | Parameter pruning, model fine-tuning, watermark overwriting                                                          |
| [66]   | To embed watermark in model parameters                                  | Parameter regulariser                          | CIFAR10, Caltech-1101             | WRN                          | Train-to-embed     | No         | Fidelity, TER, BER, Robustness, capacity                  | BER is 0 and TER is low. Low embedding loss.             | Weak against watermark overwriting attacks.          | One of the convolution layer            | Bit vector                                                      | Binary cross entropy                    | Fine tuning, parameter pruning, distillation attack, watermark overwriting                                           |

(continued on next page)

Table 4 (continued)

| RefNo. | Objective                                                        | Technique                              | Datasets                              | Model                                                | Embedding          | Reversible | Performance Metric                              | Results                                                                                           | Limitations                                                              | Embedding Location                                                           | Type of watermark           | Loss function                                                                                      | Attacks                                                                                                                       |
|--------|------------------------------------------------------------------|----------------------------------------|---------------------------------------|------------------------------------------------------|--------------------|------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------|-----------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| [67]   | To develop a DNN watermarking scheme using DM-QIM technique      | DM-QIM                                 | Flower dataset                        | VGG16                                                | Fine-tune-to-embed | No         | Accuracy and loss, secrecy and robustness       | High accuracy, very small loss. Secure against parameter pruning.                                 | Model accuracy degrades after watermark embedding                        | Fully connected layer, in weights                                            | Bit vector                  | Binary cross entropy                                                                               | Watermark overwriting, pruning                                                                                                |
| [68]   | A passport-aware normalization formulation for IP protection     | Passport-aware normalization           | CIFAR10, CIFAR100, ModelNet, SharpNet | AlexNet, ResNet, PointNet                            | Train-to-embed     | No         | Accuracy, robustness                            | Secure against fine-tuning, parameter pruning and watermark overwriting. Slightly better accuracy | Replacing BN with GN cause great performance drop                        | Added new passport aware normalization layer parallel to normalization layer | -                           | Sum of Cross entropy, trigger-set based IP protection loss, passport signature regularization loss | Fine-tuning, parameter pruning, ambiguity attack,                                                                             |
| [49]   | To develop a reversible watermarking technique for IP protection | Histogram Shift for embedding. SHA-256 | ImageNet                              | AlexNet, VGG19, ResNet152, DenseNet121 and MobileNet | Train-to-embed     | Yes        | Fragility, Reversibility, Robustness, Capacity  | Low accuracy, exhibits degradation                                                                | Low accuracy, exhibits degradation                                       | Convolution layer, in weights                                                | Bit vector                  | Cross entropy                                                                                      | No attacks discussed, Accuracy Comparison with non-reversible models                                                          |
| [69]   | To develop a Reversible watermarking technique for IP protection | Quantization Index Modulation          | MNIST, CIFAR-10                       | MLP, VGG                                             | Train-to-embed     | Yes        | Usability test, capacity and fidelity, accuracy | High capacity, High accuracy                                                                      | Not applicable for Black-Box                                             | In weights                                                                   | Bit vector                  | -                                                                                                  | Integrity protection, infringement identification                                                                             |
| [70]   | To develop a highly robust DNN watermarking scheme               | Encoder-decoder network                | MNIST, IMDB, CIFAR10, CIFAR100        | VGG11, GoogLeNet, ResNet18, ResNet24                 | Train-to-Embed     | No         | Accuracy, robustness, training time             | Robust against kernel cutoff, transfer learning and pruning attacks                               | Takes large time to embed watermark. Not robust against model extraction | Parameter space of DNN model                                                 | Encoder part of autoencoder | Loss function                                                                                      | Fine-tuning, transfer learning, pruning, kernel cutoff/supplement, channel recording, channel expansion, parameter adjustment |

**Watermark Embedding****Watermark Detection**

**Fig. 10.** Black-box watermark embedding and verification process.

watermark can be extracted with perfect accuracy even after pruning 99 % of model parameters with zero false positive and false negative rates and is immune to watermark removal techniques. In [64], the author proposed a fingerprinting framework for IP protection and digital rights management with two methodologies: orthogonal fingerprinting and coded fingerprinting, which embeds fingerprints in the density probability distribution of weights. The approach was evaluated on MNIST and CIFAR10 datasets on CNN and wide residual networks and shows no accuracy drop after embedding and a high robustness of  $\sim 99\%$  against parameter pruning and fine-tuning. In [65], the author proposed a generic watermarking methodology for both white-box and black-box approaches. The owner can embed the watermark during the model's fine-tuning phase using the following three steps. First, an  $n$ -bit binary random string is generated and embedded in the probability density function distribution of different layers, followed by the generation of specific input keys that can trigger the watermark after embedding. Finally, the neural network is trained with constraints imposed by the watermark after embedding. The watermark can be extracted in both black-box and white-box settings. The results showed that the scheme has high accuracy and is reliable and robust. A general framework to embed a watermark of size 256 bits in model parameters of the second convolution layer using a parameter regulariser was proposed in Nagai et al. [66]. Experiments were conducted on wide residual networks (WRN) using CIFAR-10 and Caltech-101 datasets. The host network was trained from scratch (train-to-embed) to embed the watermark into the model parameters. The results revealed that the BER value is zero, and the TER value is significantly low in all the cases. The test error for the embedded model is close to that of a clean model. Embedding loss increased slightly after fine-tuning. The watermark persists even after 65 % of the parameters are pruned. It is observed that the scheme has a poor defence against watermark overwriting attacks. A DNN watermarking scheme utilising the dither modulation-quantization index modulation technique to embed a watermark in network weights was proposed in Kuribayashi et al. [67], where dither modulation operation was used to enhance secrecy. The embedding operation is performed on the frequency components of sampled weights using the fine-tune-to-embed approach, in which the embedding fully connected (FC) layer is implanted after the final convolution layer; the weights of the DNN model are gently adjusted in each epoch. The weights at FC layers are targeted for embedding watermarks. Experiments were conducted on the VGG16 model, which was trained on the ImageNet dataset and fine-tuned on the Flower dataset provided by VGG. The results exhibited that the watermark can be recovered with perfect accuracy for 50 % or less pruning. Overwriting attacks greatly degrade model accuracy and loss. The advantage of the method is its low distortion level. A passport-aware normalisation formulation where a passport-aware branch is added for IP protection was proposed in Zhang et al. [68]. The model was trained, and the passport-aware branch was removed to cause no change in the model structure. When verifying ownership, the passport-aware branch was again added. Even after 90 % parameter pruning, more than 90 % of signatures could be detected.

The challenge with all the approaches mentioned above is that the embedded watermark violates the integrity of the embedded model and permanently reduces the original task accuracy because the watermark is permanently embedded in the model parameters

Table 5

State-of-the-art black-box watermarking techniques.

| Ref No. | Objective                                                                        | Technique                      | Datasets                               | Model                                           | Embedding                          | Performance Metric                                                 | Results                                                                              | Limitations                                         | Embedding Location                                   | Type of watermark | Loss function                                                  | Attacks                                                                                                                                                                             |
|---------|----------------------------------------------------------------------------------|--------------------------------|----------------------------------------|-------------------------------------------------|------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------|-------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [27]    | To design robust watermarking algorithm                                          | Backdoor                       | CIFAR-10, CIFAR-100, ImageNet          | ResNet-18, CNN with 18 layers                   | Train-to-embed, fine-tune-to-embed | Accuracy, Fine-tuning                                              | High accuracy                                                                        | No data on watermark overwriting attacks            | DNN's learned parameters                             | Image             | Negative log likelihood                                        | Fine-tune last layer, fine-tune all layers, Re-Train last layer, Re-Train all layers, transfer learning                                                                             |
| [71]    | To develop an invisible watermark for ownership verification                     | GAN, MSE, BCE loss             | CelebA, FFHQ, bedroom and VGG flowers. | BEGAN, PGGAN, StyleGAN2.                        | Fine-tune-to-embed                 | PSNR, watermark bit accuracy, robustness                           | High PSNR and SSIM, high watermark bit accuracy.                                     | Watermark capacity is 100 bits which is very low.   | Output images                                        | Bit string        | Sum of Mean squared error (MSE) and Binary cross entropy (BCE) | JPEG compression, noise addition, blurring, color transformation                                                                                                                    |
| [72]    | To provide sustainable and proactive solution for deep fake detection            | Fingerprinting                 | CelebA, LSUN cat, CIFAR-10             | ProGAN, StyleGAN, StyleGAN2, BigGAN, CUT        | Train-to-embed                     | Transferability, Fidelity, Robustness, Secrecy, Deepfake detection | High model accuracy                                                                  | Need to analyse tolerable pruning                   | Output images                                        | Image             | Sum of Mean squared error (MSE) and Binary cross entropy (BCE) | Additive Gaussian noise, blurring with Gaussian kernel, JPEG compression, centre cropping, model weight quantization, adding Gaussian noise to model weights.                       |
| [73]    | To develop a black box watermarking scheme for pre-trained models                | Backdoor                       | MNIST, CIFAR-10, ImageNet.             | CNN                                             | Fine-tune-to-embed                 | Fidelity, Robustness, integrity, capacity, overhead                | High accuracy, low false positive, high capacity                                     | Weak against watermark overwriting attacks.         | Distributions of output activations (before softmax) | Arbitrary string  | Hamming distance, cross-entropy loss                           | Model fine-tuning, model pruning, watermark overwriting                                                                                                                             |
| [74]    | To develop a zero-bit watermarking algorithm.                                    | Fast Gradient Sign method      | MNIST                                  | MLP, CNN, IRNN                                  | Fine-tune-to-embed                 | Fidelity, robustness, false positive rate                          | High accuracy, low degradation. No false positive.                                   | Model accuracy degrades after watermark embedding   | Tweaking decision frontier                           | Bit string        | -                                                              | Model pruning, singular value decomposition, overwriting via fine-tuning                                                                                                            |
| [75]    | To address problem of IP theft via model extraction                              | HMAC with SHA-256              | MNIST, Caltech, GTSRB, CIFAR-10        | DenseNet-121, ResNet-34                         | Fine-tune-to-embed                 | Robustness, capacity, efficacy                                     | High accuracy, low TER, hard to remove watermark. Immune to extraction.              | -                                                   | Subset of queries received                           | Bit string        | -                                                              | Model extraction, distributed model extraction, double extraction, fine-tuning, pruning, training with noise, adding noise during inference, recognizing queries from training data |
| [76]    | To develop a black box watermarking protocol against forging attack              | SHA-256                        | CIFAR-10, CIFAR-100                    | ResNet-18                                       | -                                  | Accuracy                                                           | High classification accuracy.                                                        | -                                                   | Trigger set                                          | Image             | -                                                              | Forging attack, transfer learning                                                                                                                                                   |
| [77]    | To watermark DNN model trained using federated learning                          | -                              | MNIST, CIFAR10                         | 5-layer CNN, VGG-16                             | -                                  | Accuracy, robustness                                               | High model accuracy. Secure against parameter pruning.                               | -                                                   | Different layers of DNN models                       | Image             | Cross-entropy                                                  | Fine-tuning, pruning, neural cleanse, ownership piracy, model stealing                                                                                                              |
| [78]    | To watermark deep learning models and verify using remote verification mechanism | -                              | MNIST, CIFAR10                         | DNN models                                      | Train-to-embed                     | Accuracy, robustness                                               | High watermark accuracy. Secure against model pruning and fine-tuning.               | Watermark cannot be detected in some scenarios.     | Input images (fundamental functionality of model)    | Image             | Cross-entropy                                                  | Model pruning, fine-tuning, inversion attack,                                                                                                                                       |
| [79]    | A new DNN watermarking framework suitable for Embedded applications.             | Digital signatures             | MNIST, CIFAR10                         | LeNet, VGG-16, ResNet-50, DenseNet-121          | Fine-tune-to-embed                 | Accuracy, False positive, false negative rate                      | High accuracy. False positive and false negative rate is close to original model.    | -                                                   | Input images (fundamental functionality of model)    | Bit string        | -                                                              | Tampering, Ghost Signature attack                                                                                                                                                   |
| [80]    | To develop Persistent and piracy resistance watermarking                         | Null embedding, wonder filters | Digit, Traffic, Face datasets          | 2conv+2Dense, 6conv+2Dense, 4conv+1merge+1Dense | Fine-tune-to-embed                 | Accuracy, false positive, robustness, confidentiality              | False positive rate is 0. Secure against Fine-tuning. Confidentiality is maintained. | Prior information of watermark. Not tested on large | Input images (fundamental functionality of model)    | Bit string        | Cross-entropy                                                  | Fine-tuning, Neuron pruning, piracy, Transfer learning, Detect existing watermark, corrupt                                                                                          |

(continued on next page)



Table 5 (continued)

| Ref No. | Objective                                                                          | Technique                                     | Datasets                                                | Model                                                                              | Embedding          | Performance Metric                     | Results                                                                                                                          | Limitations                                                                                    | Embedding Location                                                                           | Type of watermark | Loss function                                                                                          | Attacks                                                                                                                                                |
|---------|------------------------------------------------------------------------------------|-----------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------|--------------------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| [81]    | To develop watermarking scheme which is secure against forgery using fake triggers | Frequency domain image watermarking technique | MNIST, CIFAR10, CIFAR100                                | LeNet-5, VGG-13,16,19, ResNet-18,34,50,101                                         | Train-to-embed     | Accuracy, False alarm rate, Robustness | Slightly degrade in accuracy. Zero false alarms. Secure against pruning.                                                         | datasets Limited capacity. Watermark overwriting not discussed and might remove the watermark. | (embedding 2D wonder filter into the image Input images (fundamental functionality of model) | Bit string        | Cross-entropy                                                                                          | known watermark, Embedding a new watermark Piracy, transfer learning, fine-tuning, neuron compression, neural cleanse, fine pruning, model extraction, |
| [82]    | To protect copyright and improve detectability of DNN watermark                    | Reversible image hiding network               | MNIST, Fashion-MNIST, CIFAR-10, CIFAR-100, Caltech-101  | LeNet-3, LeNet-5, VGG-11, VGG-13, VGG-16, VGG-19, ResNet-18, ResNet-34, ResNet-101 | Train-to-embed     | Accuracy, SSIM, PSNR                   | Low accuracy degradation, High PSNR, SSIM value                                                                                  | Does not discuss defence against major attacks against watermarks                              | Fundamental functionality of the model                                                       | Image             | Mean squared error, binary cross-entropy, structure similarity index measure(SSIM), cross-entropy loss | -                                                                                                                                                      |
| [83]    | To verify copyright                                                                | U-net structured encoder                      | CIFAR-10, GTSRB, Tiny-ImageNet                          | ResNet-18                                                                          | Train-to-embed     | Accuracy, SSIM, PSNR, MSE, LPIPS       | High accuracy, immune to fine-tuning, parameter pruning                                                                          | Robustness degrades if fine-tune-to-embed method is used                                       | Fundamental functionality of the model                                                       | Image             | Cross-entropy loss, triple loss                                                                        | Fine-tuning, weight pruning, JPEG compression, neural cleanse                                                                                          |
| [84]    | To be immune against removal attack                                                | Feature fusion                                | MNIST, CIFAR-10, CIFAR-100, Tiny-ImageNet               | LeNet-5, VGG16, ResNet-18                                                          | Train-to-embed     | Accuracy, efficiency, fidelity         | Immune to fine-tuning, transfer learning, weight pruning and watermark overwriting attacks                                       | No way to verify model ownership                                                               | Model functionality                                                                          | Image             | -                                                                                                      | Fine-tuning, transfer learning, weight pruning and watermark overwriting attacks                                                                       |
| [85]    | To efficiently embed watermark into DNN models                                     | -                                             | CIFAR-10, CIFAR-100, ImageNet                           | ResNet-18                                                                          | Train-to-embed     | Accuracy                               | Immune to functionality stealing, soft-label model extraction, hard-label model extraction, fine-tuning and fine-pruning attacks | -                                                                                              | Model functionality                                                                          | Image             | Binary cross-entropy                                                                                   | Functionality stealing, soft-label model extraction, hard-label model extraction, fine-tuning and fine-pruning attacks                                 |
| [86]    | To propose a DNN watermarking scheme that is immune to ambiguity attacks           | -                                             | MNIST, CIFAR-10, CIFAR-100, FOOD-101                    | ResNet18, WRN28_10, EfficientNet_B0                                                | Fine-tune-to-embed | Test accuracy, trigger accuracy        | Robust to ambiguity attack and brute force attack.                                                                               | Defence against major attacks against watermarks not discussed                                 | Model functionality                                                                          | Image             | Cross-entropy                                                                                          | Ambiguity attack, brute force attack.                                                                                                                  |
| [87]    | To propose a black box multi-bit DNN watermarking scheme                           | Power function mapping                        | CIFAR-10, CIFAR-100, Caltech-101, Caltech-256, FOOD-101 | ResNet-18, GoogLeNet, ShuffleNet                                                   | Train-to-embed     | Accuracy                               | High accuracy, very low BER                                                                                                      | Watermark overwriting attacks and security of secret key is not evaluated                      | Final softmax output layer of the DNN                                                        | Bit string        | Cross-entropy                                                                                          | Fine-tuning, pruning and parameter quantization                                                                                                        |

and cannot be removed, even after the watermark extraction process. To address this issue of integrity authentication, reversible watermarking in Deep CNN was proposed in Guan et al. [49]. The author used the histogram shift method to embed the watermark. In this method, first, the host sequence is constructed. Data is converted to integers to make it feasible with the watermarking scheme, and last, the watermark is embedded using the histogram shift technique. Tests were performed using the ImageNet dataset on VGG19, ResNet152, DenseNet121 and MobileNet models to evaluate performance; the tests indicated that model accuracy remains the same even after watermark embedding, classification accuracy is also identical to un-embedded models and has 0 % reconstruction error rate. However, there is low accuracy with floating point weights, which exhibits degradation when host exhibits uniform-like distribution. To manage the challenges with this method, reversible quantisation index modulation for static deep neural network watermarking was deployed in a study [69]. The process utilises the quantisation index modulation technique to embed a watermark into the model in a reversible manner. Experiments were conducted using the MNIST and CIFAR-10 datasets on MLP and VGG models. Results demonstrated higher capacity and accuracy than the above method. Integrity protection can be achieved by comparing the restored and original models. In [70], the author proposed a DNN watermarking scheme, which is highly robust against fine-tuning, transfer learning, kernel cutoff/supplementation, functionality-equivalent attacks and fraudulent ownership claims. The scheme operates in three phases: watermark generation, embedding and extraction. In watermark generation, an encoder-decoder network is trained on a small dataset and divided into encoder and decoder sections. The encoder part is embedded as a watermark in the original neural network with the training data, and the decoder part is kept secret by the model owner. To detect an IP infringement, the owner first extracts the encoder part from the suspect model, then merges it with the decoder part and then tests it on a small dataset. The given approach was evaluated on different models using standard datasets. It was observed that training time increased significantly. The results indicated robustness against kernel cutoff, transfer learning and pruning attacks, but the scheme is not robust against model extraction attacks.

A summary of some notable white-box watermarking schemes is listed in Table 4. When considering white-box watermarking, the above state-of-the-art analysis reveals the following key points:

- White-Box watermarking schemes are easy to implement, are less time-consuming, and provide sufficient security against parameter pruning, model fine-tuning and watermark overwriting attacks.
- One of the major drawbacks is that the users can verify the IP theft only if they have the model and can access the model's parameters; the watermark is embedded into the weights or activations and can only be extracted directly from weights or activations.
- Nowadays, model owners upload their models on the cloud as machine learning as a service (MLaaS); white-box watermarking is completely ineffective in this scenario.

### 2.1.2. Black-box watermarking

The white-box watermarking scheme discussed above requires the user to have direct access to the internal structure of the model to extract the watermark from the model parameters; however, this is not always possible, considering the developers can host a pre-

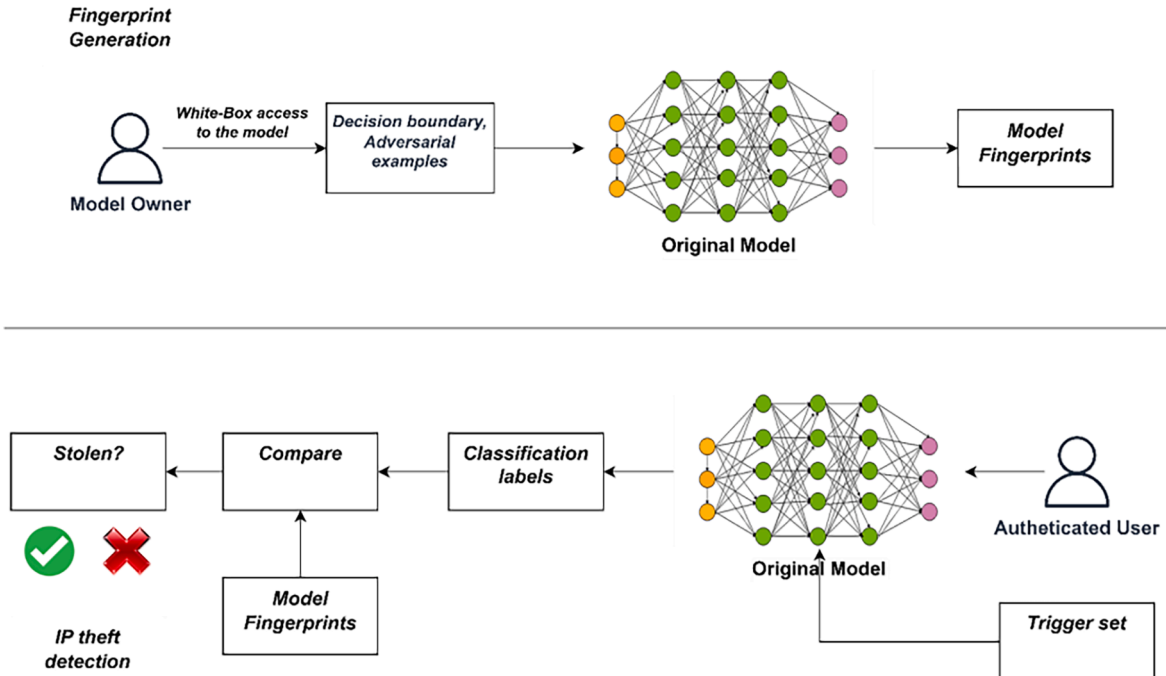


Fig. 11. A general approach to fingerprinting generation and IP theft detection.

trained model on their own servers or the cloud as MLaaS. In this situation, the user will not be able to extract the watermark. For this reason, black-box watermarking has been proposed as a solution [27]. The model owner creates a perturbed set of data called the trigger set. Trigger sets are the images which, when queried, misclassify and produce the wrong label. The process of black-box watermark embedding, and verification is illustrated in Fig. 10. The model owner embeds the watermark into the original model either at the time of training by training the model on the training set and trigger set or at the time of fine-tuning the trained model, by fine-tuning the model using the trigger set. The authorised user cannot directly access the model and cannot check model parameters. To verify suspect model ownership and detect IP theft, an authorised user queries the suspect model using a trigger set; the results are compared to the model owner's results. If the user receives the same wrong label output as from the owner's model, then it may be concluded that the suspect model is a copy of the original model. The user cannot directly access the weights and parameters of the DNN model; to overcome this problem, the Turning your Weakness, a black box watermarking technique, was proposed in Adi et al. [27]. Backdooring is a technique to train neural networks to output wrong labels for certain inputs, called trigger sets. The method was evaluated using three standard datasets on the ResNet-18 model. Results highlighted that the scheme is immune to fine-tuning, parameter pruning and transfer learning while retaining the same accuracy as the unembedded model. The extent of pruning up to which this scheme is robust is yet to be examined. In [71], the author proposed a watermarking scheme to protect the IP of generative adversarial networks (GAN). First, the watermarking network was trained, and then the decoding part of the network was frozen and exploited to train the GAN to enforce the injection of watermarks inside the generated images. Subsequently, the generator was optimised to minimise the combined loss of watermark and GAN. Results revealed a bit accuracy of 100 %, as well as high peak-signal-to-noise ratio (PSNR) and structure similarity index (SSIM) values. However, the watermark capacity is 100 bits, which is substantially low. In [72], the author introduces artificial fingerprinting into generative models to detect deepfakes. The fingerprint embedding was performed on the well-trained encoder(E) and decoder(D) networks. The results demonstrated excellent bit accuracy and robust image and model perturbation; deepfakes could be detected in all the cases. In [73], the author proposed a black-box watermarking technique for protection against IP infringement. First, the watermark keys were generated, and the model was fine-tuned such that an owner-specific signature was encoded in the output activations. To verify the ownership, the model was queried with watermark keys for the model output owner's signature. The method used the MNIST, CIFAR10 and ImageNet datasets with three network architectures. Results demonstrated that fidelity was low, and the model was robust against fine-tuning and parameter pruning, as even after 95 % of parameter pruning, a watermark could still be detected. It was also robust against watermark overwriting with zero BER and zero false positive and false negative rates. A study propounded a novel zero-bit watermarking algorithm using adversarial model examples [74]. It followed the black-box approach to extracting watermarks using only a few queries. Effectiveness is required for this approach instead of capacity. The fast gradient sign method was used to create adversaries. Experiments were performed on three different models using the MNIST dataset, discovering high accuracy and extremely low degradation rates. There were no false positives, and the accuracy of extracting the watermark after model pruning was high. For watermark overwriting, there was perfect watermark extraction for CNN and MLP, while there were a few failures for IRNN. The author proposed a DNN watermarking scheme to address the challenge of IP theft via model extraction by training a surrogate model using the results from the prediction API of the original model [75]. The HMAC code with the SHA-256 function was used to generate a random secret key. Experiments were performed on low-capacity (DenseNet121) and high-capacity models (ResNet34) using four standard datasets. Several surrogate models were trained and evaluated using Dropout and weight decay regularisers, demonstrating a high impact on the test and watermark accuracy. Results revealed low TER and remarkably few watermark removal cases. The author proposed a black-box watermarking protocol against black-box forging attacks using a one-way hash function (SHA-256) [76]. Trigger samples were used to prove ownership from a one-way chain, and their labels were also assigned. Tests were conducted on ResNet18 using CIFAR-10 and CIFAR-100 datasets. The results showed high classification accuracy and considerably low loss compared to the unmarked model. The method is robust, as the owner's trigger samples were correctly classified with high accuracy in most cases. Backdooring requires full access to training data and control of the training process, which is not possible in federated learning as training data is distributed among several client devices. To address this challenge, the author proposed an approach to watermark the DNN model trained using federated learning [77]. The model was watermarked by adding specific patterns to images containing only noise. Several images were generated using Gaussian noise; each image was embedded with certain patterns and labelled with a class related to the original task. A watermark was embedded at each aggregation round of federated learning. The procedure is independent of training data and is suitable for federated learning. Tests were performed on CNN and VGG16 models using two standard datasets, and 100 clients participated in the training. Results revealed that model test and watermark accuracies were high; however, the test accuracy of the original task started to decline after half of the neurons were removed. A study analysed a watermark implanting approach to infuse watermarks into DL models and design a remote verification mechanism [78]. The watermark was embedded while training the model from scratch. Three watermark generation algorithms were investigated: using meaningful content embedded in original training data as watermarks, independent training data with unrelated classes as watermarks, and pre-specified noise as watermarks. The generated watermarks were embedded in the model. Experiments conducted on various DNN models using MNIST and CIFAR10 datasets revealed better accuracy than the clean model and high watermark detection accuracy even when 90 % of the parameters were pruned; test accuracy and watermark accuracy remained almost unchanged after fine-tuning. It is impossible to detect the watermark if the leaked model is used as an internal service, and the ownership cannot be verified if the verification queries are prevented. The author proposed a DNN watermarking framework suitable for embedded applications in Guo and Potkonjak [79]. A watermark can be embedded by fine-tuning the trained model with a trigger set created by mapping the owner's signature to modify a portion of the DNN training set. It does not consider transfer learning and model compression as threats. The approach was evaluated on various DNN models using three standard datasets. Results revealed that when a 128-bit message mark is embedded, very low accuracy drops are recorded, and the false positive and false negative rates are comparable to the original model. The author believes

that the properties of persistence and piracy resistance are critical to watermarks and proposes two training techniques: out-of-bound values and null-embedding [80]. The author also introduced a new primitive that embeds a persistent bit-sequence into a model. Out-of-bound values are pixel values larger than normal input values, resulting in classification rules that recognise target labels with perfect certainty. Null embedding can only be trained at initial training and cannot be added to a trained model without hampering its accuracy. A pixel-based primitive was used to persistently encode a bit sequence into a DNN model by utilising out-of-bound values and null embedding. A watermark was generated using the owner's private key for ownership verification. Experiments were conducted on CNN models using Digit, Traffic, and Face datasets. Results indicated that watermarking causes a slight degradation in the classification accuracy for all the tasks, and the false positive rate is zero. Fine-tuning cannot remove the watermark from the model, and to maintain confidentiality, watermark information is stored in a secure server. The approach has not been tested on large datasets, and the watermark size is observed to be limited. An informed attacker can forge a fake trigger and claim ownership of the model; to avoid this, the author proposed a watermarking approach exploiting frequency domain image watermarking to generate triggers and build a DNN watermarking algorithm [81]. Due to the robustness of frequency domain image watermarking, attackers cannot forge triggers easily. Trigger sets can be created by inserting a block-based feature pattern in the frequency domain. The dataset and trigger set were merged to form a new training set, and the model was trained on it to embed the watermark into the model. Experiments were performed on multiple models using CIFAR10 and CIFAR100 datasets. The results indicated that, in most cases, the accuracy after embedding remains unchanged compared to the clean models, and no false alarms are reported. Even after pruning 80 % of the model parameters, the watermark could be extracted with 100 % accuracy. The author did not discuss watermark overwriting attacks; trigger removal attacks might remove the watermark. To protect the owner's copyright and improve the watermark's detectability, the author proposed a dynamic DNN watermarking technique based on a reversible image-hiding network [82]. The framework comprised three parts: a reversible image hiding network, which generates key samples by training the network on a subset training set and the secret image that is hidden into the cover image; a discriminator, which ensures that the distortion of the cover image is minuscule after hiding the secret image; and the original host network, which is the original owner's model to be watermarked. The original host network was trained on ordinary and key samples to get a watermark embedded in the host network. To confirm the ownership of the model, the user must query the remotely deployed model with key samples to obtain predictions. The secret image can be perfectly recovered from the revealing block of the reversible image-hiding network. Experiments were conducted on multiple models using five standard datasets. The results revealed that the approach showcases minimally low accuracy degradation after watermark embedding and high PSNR and SSIM values. For copyright verification, the author proposed an interesting imperceptible model watermarking approach using deep hiding [83]. The proposed method consisted of three parts: embedding secret information, where a U-net structured encoder is used to embed secret images into the selected subset of a dataset to generate watermarking dataset; a training watermark classifier, where the original model is trained on the watermarking and training datasets, and a triple loss function is used to learn the triggers of the watermark; ownership verification, where the user can query the classifier prediction API using watermarking samples for a suspect model and obtain predicted labels. Watermark accuracy is calculated, and if it is greater than the threshold value, the suspect model is established as a pirated copy of the original model. Experiments were conducted on the ResNet1-8 model using CIFAR-10, GTSRB and Tiny-ImageNet datasets. The results showed high accuracy and immunity against fine-tuning, weight pruning and JPEG compression attacks. The author proposed a novel DNN watermarking solution that is effective against watermark removal attacks [84]. The proposed method contained three main modules: trigger generation, watermark embedding, and watermark verification. In the trigger generation module, function-coupled watermark triggers are generated using two feature-fusion methods: direct feature-fusion and invisible feature-fusion. In the watermark embedding module, generated triggers are mixed with training data, and the model is trained on this data to embed the watermark in the functionality of the model. In the watermark verification module, the user sends a query to the remote suspect model using watermarking samples and gets labels as a response. If the authentication success rate is higher than the threshold value, the ownership of the model stands verified. The scheme was evaluated on various models by training them on different standard datasets. The result indicated that the model has high fidelity and is robust against fine-tuning, transfer learning, weight pruning and watermark overwriting attacks. The author introduced a novel watermarking method to efficiently embed watermarks into the DNN models [85]. The approach is based on real-world datasets containing diverse features that can be exploited for classification. The trigger sets were constructed using the multi-view data. The trigger sets were trained along with clean data, thereby embedding the watermark into the model. The user can verify the ownership of the model by querying the suspect model with the trigger set and comparing the model's prediction on the trigger set. If this value is less than a certain threshold value, then the ownership of the model is established. The method was evaluated on the ResNet-18 model using three standard datasets. The results showed that the model is not affected by functionality stealing, soft-label and hard-label model extraction, fine-tuning and fine-pruning attacks.

An ambiguous backdoor watermarking scheme was tested to resolve the challenges of ambiguity attacks [86]. The trigger set was transformed into a two-dimensional one to safeguard the watermark from ambiguity attacks, making it unambiguous. The watermark was embedded into the original model by fine-tuning all layers of the model using the trigger set. The user can verify the copyright of the model by querying it with a trigger set and comparing the received label predictions. The method was evaluated on multiple models using different standard datasets. The results highlighted that the scheme is robust to ambiguity and brute force attacks. Another study proposed a black-box multi-bit DNN watermarking algorithm suitable for multiclass classification networks [87]. The watermark was embedded into the DNN model by adding a watermarking loss term to the loss used to train the model. The output was divided into groups, and each group was projected onto a secret sequence to extract the watermark. The sign of projection of groups determines the watermark bits. The approach was evaluated on three DNN models using different standard datasets. The results illustrated that the approach has good accuracy, and the BER value is very close to zero. The scheme is robust against fine-tuning, pruning and parameter quantisation attacks.

Table 6

State-of-the-art DNN fingerprinting techniques.

| RefNo. | Objective                                                                                                     | Technique                                                       | Datasets                           | Model                                             | Fingerprint Generation | Fingerprint Verification | Performance Metric              | Results                                                                                                                  | Limitations                                                                                                                | Extraction Location                                 | Type of fingerprint                        | Loss function      | Attacks                                                                                                               |
|--------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|------------------------------------|---------------------------------------------------|------------------------|--------------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------|
| [88]   | DNN fingerprinting technique that incur no accuracy loss of target classifier                                 | Classifier's classification boundary analysis, gradient descent | CIFAR-10, CIFAR-100, ImageNet      | ResNet-20, WRN-22-4, ResNet-50                    | White-Box              | Black-Box                | Accuracy, matching rate, ARUC   | Very low accuracy loss, high matching rate, AUC is 1                                                                     | Not evaluated against knowledge distillation attacks                                                                       | Near classification boundary                        | Pair of data point and its predicted label | -                  | Fine-tune last layer, fine-tune all layer, retrain last layer, retrain all layers, weight pruning, filter pruning,    |
| [89]   | DNN fingerprinting technique that is robust against model modification and model extraction attacks           | Conferring Adversarial examples                                 | CIFAR-10, CIFAR-100, ImageNet32    | ResNet-20, VGG-16/ VGG-19, DenseNet               | White-Box              | Black-Box                | Accuracy, robustness, AUC       | No loss in classifier accuracy. ROC AUC is 1.0                                                                           | -                                                                                                                          | Weights of first n layers                           | Dual modality (bit vector)                 | -                  | Fine tuning, transfer learning, non-repudiation, ownership verification                                               |
| [90]   | A DNN Fingerprint for Non-Repudiable Model Ownership Identification and Piracy Detection of fine-tuned model. | Side channel analysis                                           | CIFAR-10, CIFAR-100, ImageNet-10   | ResNet-20, VGG-11, DenseNet161                    | White-Box              | Black-Box                | Accuracy, robustness            | Effective against model fine-tuning, model extraction attack, Non-repudiation and Ownership verification can be verified | -                                                                                                                          | Model's classification boundary                     | Images                                     | -                  | Model fine-tuning, model pruning                                                                                      |
| [91]   | DNN fingerprinting approach used for IP protection and remote model ownership verification                    | Inherent features of model                                      | MNIST, CIFAR-10                    | LeNet-1, LeNet-4, Lenet-5, VGG-11, VGG-13, VGG-16 | White-Box              | Black-Box                | Accuracy                        | Fingerprint visible even after 90 % of parameters are pruned.                                                            | Only focused on classification tasks. Computational overhead not discussed.                                                | Model's classification boundary                     | Owner's training samples (images)          | -                  | Fine-tune last layer, fine-tune all layer, pruning, transfer learning, model extraction, adversarial model extraction |
| [92]   | Fingerprinting approach to detect model stealing                                                              | Sample correlation difference, Gaussian RBF                     | CIFAR-10, CIFAR-100, Tiny-ImageNet | ResNet, DenseNet, MobileNet                       | White-Box              | Black-Box                | Accuracy, AUC-ROC               | AUC is 1.0 in most cases.                                                                                                | Performs slightly worse than other approaches. Selection of optimal model stealing detection parameter d is not discussed. | Model's classification boundary                     | Images                                     | -                  | Model pruning                                                                                                         |
| [93]   | DNN fingerprinting method with high robustness and low transferability                                        | DCT, High pass frequency mask                                   | CIFAR-10                           | VGG-16, VGG-19                                    | White-Box              | Black-Box                | AUC, F1 score, accuracy         | Low false alarm rate, high uniqueness, perfect F1 score                                                                  | -                                                                                                                          | Model's classification boundary                     | Images                                     | -                  | Fine-tune last layer, fine-tune all layer, retrain last layer, retrain all layers, model pruning, model extraction    |
| [94]   | A testing framework for copyright protection of deep learning models                                          | Jensen-Shanon distance                                          | MNIST, CIFAR-10, ImageNet          | LeNet-5, ResNet-50, VGG-16 and LSTM               | White-Box              | White-Box and Black-Box  | Accuracy, AUC                   | AUC is 1.0, immune to model modification and model extraction attacks                                                    | Computational overhead not discussed                                                                                       | Model's decision boundary (last layer of the model) | Datapoints (images)                        | -                  | Fine-tuning, model pruning, model quantization, adversarial training                                                  |
| [95]   | An accurate, general and IP protection framework against model extraction attacks                             | K-mean clustering, cosine similarity                            | FMNIST, CIFAR-10, Tiny-ImageNet    | ResNet18, ResNet34, VGG16, DenseNet121, GoogLeNet | White-Box              | Black-Box                | Accuracy, AUC                   | High accuracy, Immune to model extraction attack, AUC is 1                                                               | Overhead of training data preparation for the encoder is not discussed.                                                    | Model's unique classification boundary              | Data points (images)                       | -                  | Feature extraction, weight pruning, filter pruning, fine-tuning                                                       |
| [96]   | IP protection using DeepFool                                                                                  | DeepFool                                                        | CIFAR-10, CIFAR-100, ImageNet-10   | ResNet18, ResNet20                                | White-Box              | Black-Box                | Testing accuracy, Matching rate | 100 % detection rate, 0 % false positive rate                                                                            | Does not discuss defence against collusion attacks and transfer learning.                                                  | Model's classification boundary                     | Images                                     | Cross-entropy loss | Compression attacks, unstructured pruning, irregular pruning,                                                         |

(continued on next page)

Table 6 (continued)

| RefNo. | Objective                                                                                             | Technique                                         | Datasets                       | Model                              | Fingerprint Generation  | Fingerprint Verification | Performance Metric                                                  | Results                                                                                               | Limitations                                                                                                          | Extraction Location                            | Type of fingerprint | Loss function | Attacks                                                                                                                                                                    |
|--------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------|--------------------------------|------------------------------------|-------------------------|--------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------|---------------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [97]   | DNN fingerprinting technique for functionality verification of DNN models implemented on edge devices | Cross-entropy loss, projected gradient loss (PGD) | CIFAR-10, SVHN, ImageNet       | CNN, VGG-16                        | White-Box               | Black-Box                | Intrinsic score, accuracy                                           | Intrinsic score is 100 %                                                                              | -                                                                                                                    | Model's inner decision area                    | Images              | -             | column pruning, pattern pruning<br>Input modification, fine-tune last layer, fine-tune all layer, re-train last layer, re-train all layer, weight pruning, weight noising, |
| [98]   | A robust neural fingerprinting method for IP protection                                               | Kullback-Leibler divergence                       | CIFAR-10, Tiny-ImageNet        | ResNet, VGG, InceptionV3, DenseNet | White-Box               | Black-Box                | Accuracy                                                            | High accuracy, Immune to model modification attack                                                    | -                                                                                                                    | Model's classification boundary                | Images              | -             | Weight pruning, fine-tune last layer, fine-tune all layer, re-train last layer, re-train all layer, model distillation                                                     |
| [99]   | Task-agonistic fingerprinting framework for IP protection                                             | Adam optimizer                                    | DermaMNIST, IWPC, Fashion      | ResNet-18, CNN, MLP, DCGAN         | White-Box               | Black-Box                | Robustness, Uniqueness, Area under the Robustness-Uniqueness curve. | High true positive and true negative rate                                                             | Construction time for fingerprint is more than other fingerprinting approaches                                       | Model's decision boundary                      | Images              | -             | Transfer learning, weight quantization, model pruning, knowledge distillation, model stealing                                                                              |
| [100]  | A testing-based approach to deep learning model similarity comparison to detect model reuse.          | Distance Decision Vector, Cosine similarity       | ImageNet, Flower102, Dog120    | ResNet18, MobileNetV2              | White-Box and Black-Box | Black-Box                | Test accuracy, similarity score                                     | Effectively detects copied model from transfer learning, model compression and model stealing attacks | Framework is not evaluated on all model reuse attacks. Evaluated on CNN based models. No way to verify IP ownership. | External features                              | Images              | -             | Fully accessible attack, dataset-accessible attack, model-accessible attack, query-only attack                                                                             |
| [101]  | Fingerprinting approach to detect model integrity breach with high accuracy                           | Active Neuron cover                               | CIFAR-10, GTSRB, AT&T          | CNN, VGG-16, MLP                   | White-Box               | Black-Box                | Accuracy, False positive rate                                       | High accuracy, zero false positive, immune to model fine-tuning and parameter pruning attacks         | Does not cover security approach against transfer learning and collusion attacks.                                    | Transformed inputs                             | Images              | -             | Neural network trojan attack, error-generic poisoning, error-specific poisoning, model compression, arbitrary weight modification                                          |
| [102]  | First task-agnostic fingerprinting algorithm for DNN fingerprinting                                   | ReLU, linear regions                              | DermaMNIST, IWPC, FashionMNIST | ResNet-18, FCN                     | White-Box               | White-Box and Black-Box  | ARUC, accuracy                                                      | Immune to fine-tuning retraining, parameter pruning attacks, High ARUC value                          | -                                                                                                                    | Linear regions in input space of DNN with ReLU | Pair of images      | -             | Fine-tune last layer, fine-tune all layer, re-train last layer, re-train all layer, weight pruning, filter pruning                                                         |

Table 5 collates notable observations from original state-of-the-art papers regarding their objectives, techniques used to achieve those objectives, datasets, models, embedding as well as major performance metrics and experimental outcomes.

The above analysis highlights the following key points about black-box watermarking:

- Black-box watermarking techniques are highly effective when the DNN model is hosted on the private server of the model owner or the cloud as MLaaS, and the user does not have access to the internal parameters of the model.
- Usually, model fine-tuning is used to embed a watermark into the original model.
- As the watermark is embedded based on a triggered set, the chances of an attacker perturbing the test set to recreate the trigger set cannot be dismissed.
- A watermark, once embedded, cannot be removed.
- If the attacker can access the original training data, they can fine-tune the model with their trigger set to overwrite the original owner's watermark without significantly compromising the original task accuracy.

### Model Encryption

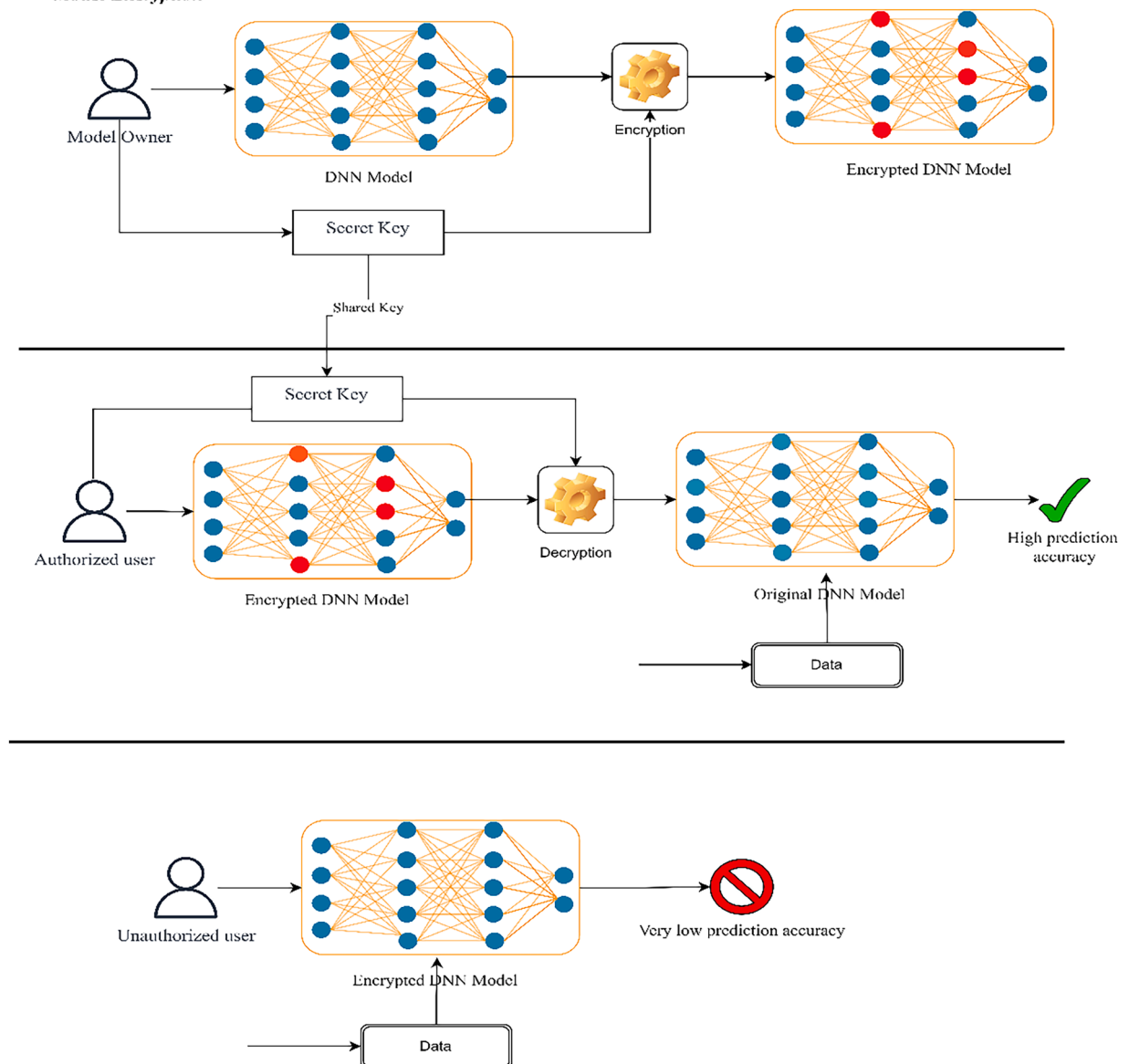


Fig. 12. A general approach to model encryption for preserving privacy of DNN model.



**Table 7**

State-of-the-art DNN encryption techniques.

| Refno. | Objective                                      | Technique                           | Datasets                       | Model                                                | Role of encryption          | Performance Metric        | Results                                              | Limitations                                            | Encryption Location                                                 | Type of data | Loss function                                         | Attacks                                                                                                            |
|--------|------------------------------------------------|-------------------------------------|--------------------------------|------------------------------------------------------|-----------------------------|---------------------------|------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------|--------------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| [104]  | To protect DNN intellectual property           | Adversarial Perturbation            | MNIST, CIFAR-10, GTSRB         | DenseNet, ResNet, AlexNet                            | Parameter encryption        | Test Accuracy             | Immune to fine-tuning, model pruning attacks         | Accuracy degradation, No way to verify model ownership | Weights of randomly selected convolution and fully connected layers | Images       | Cross-entropy loss                                    | Model fine-tuning attack, model pruning attack, adaptive attack                                                    |
| [105]  | To protect DNN models form unauthorized access | Distribution preserving random mask | CIFAR-10, ImageNet             | VGG19, DnCNN                                         | Parameter encryption        | Denoising                 | Immune to denoising, retraining attacks              | No ownership verification                              | Convolution layer                                                   | Images       | Negative cross-entropy or negative mean squared error | Denoising via wavelets, denoising via filters, re-train all layers, transfer learning                              |
| [106]  | To preserve privacy of DNN models              | Fully Homomorphic Encryption        | MNIST, CIFAR-10, GTSRB, EMNIST | LeNet-5, AlexNet, VGG16, ResNet18                    | Neurons/ filters encryption | Accuracy                  | Protects both model and data privacy                 | High computation cost, No ownership verification       | Neurons in convolution layer                                        | Images       | -                                                     | Model recovery attack, sample inference attack, weight stealing attack                                             |
| [107]  | To defend against deep learning IP theft       | Model Obfuscation                   | ImageNet                       | AlexNet, ResNet18, ResNet50                          | Filter encryption           | Accuracy                  | Robust, avoid information leakage                    | No ownership verification                              | Filters in convolution layer                                        | Images       | -                                                     | Brute-force attack, genetic algorithm attack,                                                                      |
| [103]  | To protect intellectual property of DNN        | Arnold's cat map                    | ImageNet ILSVRC-2012, CamVid   | VGG16, ResNet18, ResNet101, GoogLeNet, Unet, LinkNet | Weight encryption           | Accuracy, Inference speed | Low accuracy loss, small impact on inference speed   | No ownership verification                              | Weights of convolution layers and fully connected layer             | Images       | -                                                     | Brute-force attack, model retraining attack, model fine-tuning attack, secret key leak attack, side-channel attack |
| [108]  | To protect model and preserve privacy          | Pixel shuffling, bit flipping       | CIFAR-10                       | ConvMixer                                            | Parameter encryption        | Accuracy                  | Robust against random key attack                     | Computationally intensive                              | Convolution layer                                                   | -            | -                                                     | Random key attack                                                                                                  |
| [109]  | To protect model                               | Block-wise pixel shuffling          | CIFAR-10                       | ResNet18                                             | Model protection            | Accuracy                  | Similar accuracy, robust against fine-tuning attacks | No way to secure key                                   | Input images                                                        | -            | -                                                     | Fine-tuning attack                                                                                                 |

## 2.2. Model protection using fingerprinting

Watermarking embeds the watermark into the model parameters or activation functions and modifies the internal structure of the watermarked model, making it an intrusive approach. Fingerprinting is a non-intrusive approach to protect the IP of the model's owner [88]. A general approach to model fingerprinting is illustrated in Fig. 11. Fingerprinting functions by extracting data points near the classification boundary of the original model and using them as fingerprints. The users mostly have black-box access to the model and query the model using a trigger set to verify ownership. If an adversary copies the model, the classification boundary remains the same and hence, fingerprints can be effectively extracted, confirming the IP theft. In [88], the author observed that DNN classifiers can be uniquely represented by their classification boundary and proposed a DNN fingerprinting approach. The scheme extracts data points near the classification boundary of the model owner's classifier and uses them to fingerprint the classifier. Finding a data point near the classifier's classification boundary is an optimisation problem solved using gradient descent. The study conducted experiments on three network architectures using CIFAR-10, CIFAR-100 and ImageNet datasets. Results highlighted that classifier accuracy remains unchanged after fingerprinting, and the area under robustness-uniqueness curves (ARUC) value is 1.0 in all the cases. The author in Lukas et al. [89] examined a DNN fingerprinting technique using conferrable adversarial examples robust against all model modification and extraction attacks. First, a subclass, conferrable class, was identified; adversarial examples that maximised the output activation difference (called the conferrability score) between surrogate and reference models were determined. To accomplish this, DNN models were trained, and the example's transferability was estimated. Subsequently, the conferrable ensemble method was applied to the source model to generate an ensemble model; this model was used to generate conferrable adversarial examples by finding a perturbation delta. The error rate between the fingerprinted source and the target model was computed during the verification process. If the error rate exceeds the decision threshold (1-row), the target model is the reference model; otherwise, it is the surrogate model. Experiments established that the test accuracy remains unchanged after fingerprinting, and the ARUC value is 1.0. The approach is robust against model modification and extraction attacks but not against transfer learning attacks and adversarial training from scratch. The author used a Trusted Third party to store the timestamped fingerprint of the owner calculated on the lower-layer weights of a trained DNN model and the notary identity certificate of the model owner in Zheng et al. [90]. A notary identity certificate can be generated by a pseudo-random generator using a message digest of the model owner's notary identity. The approach was evaluated using three standard datasets on ResNet20, VGG11 and DenseNet161 models, with accuracy and robustness as the main evaluation criteria. The technique was demonstrated to be effective against model fine-tuning and extraction attacks. Non-repudiation and ownership can be proved using the owner's notary identity. To protect the IP and verify the ownership of the remote model, the author proposed a DNN fingerprinting approach [91]. The approach operates by extracting inherent features of the model and using them as fingerprints. An improved targeted attack method was used to generate unique model fingerprints by crafting specially designed adversarial examples. Experiments were conducted using MNIST and CIFAR10 datasets on different models. In order to evaluate the approach, many ghost models were trained. Test results indicated that the models have the same accuracy, and the fingerprint is highly distinguishable. In [92], the author used sample correlation (SAC) to detect model theft. Two parameters, SAC-w and SAC-m were introduced. Incorrectly classified normal samples were selected using SAC-w, and then used to detect stolen models by comparing the mean correlation between the original and stolen models. Cosine similarity and Gaussian RBF were used to determine the correlation matrix of the original and stolen models. Experiments highlighted that the approach does not hamper the accuracy of the original task; the AUC value is 1.0 for most cases, and it is neutral to fine-tuning and parameter pruning attacks. In [93], the author proposed a new DNN fingerprinting approach that used characteristic examples (C-examples), focusing on high robustness and low-transferability. The author proposed three types of characteristic examples: C-examples, robust characteristic examples (RC-examples) and low-transferability robust characteristic examples (LTRC-examples). C-examples are created from the model parameters using projected gradient descent. To obtain LTRC-examples, a frequency mask on discrete cosine transforms (DCT) is applied to construct a high pass filter. The uniqueness score is calculated to evaluate the trade-off between robustness and transferability. The model was evaluated using the CIFAR10 dataset on VGG-16 and VGG-19 models. The results showed an extremely low false alarm rate and a high uniqueness score. Perfect 1 scores were achieved for AUC and F1 scores. In [94], the author proposed a framework to quantitatively calculate the similarities between the original and the suspect DNN models. The approach comprises three steps: first, test cases (metrics) are generated for white-box and black-box settings using various dissimilarity scores. A threshold is established for each testing metric. Finally, the suspect model is assessed against each testing metric during voting where, if the distance to the original model is lower than the threshold, a positive vote is given. If the suspect model gets more positive votes, it is more likely to be a copy of the original model. Experiments were conducted using the MNIST, CIFAR-10 and ImageNet datasets on four different DNN models. Results showed that the framework achieves the AUC value of 1 for all metrics and is immune to model modification and extraction attacks.

The author proposed an accurate, robust and general IP protection framework against model extraction attacks utilising universal adversarial perturbations (UAPs) [95]. The approach works by finding UAPs and then applying K-mean clustering to cluster all training datapoints from the original model into  $n$ -clusters based on the representation vector in the model's last layer. One datapoint from each cluster is selected for comparison. An encoder is designed to detect the presence of fingerprints in a pirated model, as the pirated model will have a large cosine similarity with the original model. Experiments were conducted on multiple network architectures using three standard datasets. Results indicated that the framework has high accuracy and is immune to model extraction attacks. The AUC value is 1.0. The author proposed a fingerprinting solution for the IP protection of model owners based on the DeepFool algorithm [96]. Fingerprints were extracted around the classification boundary of the original model using the DeepFool algorithm, which adds the minimum perturbations. The owners can check if the model is theirs by querying the model with these fingerprints using a verified algorithm, which outputs a probability by applying the Bayes theorem. If this probability is less than the threshold value, the suspected

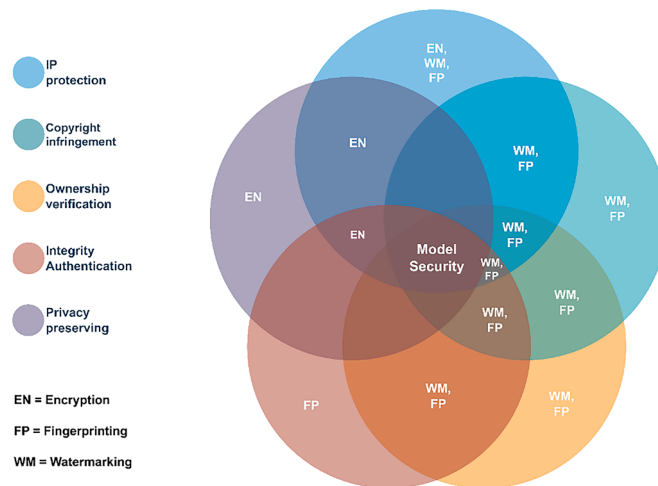


Fig. 13. Role of encryption, fingerprinting and watermarking in model security.

model is classified as the original model with high probability. Experiments demonstrated a 100 % detection rate and 0 % false positive rate. To verify the functionality of the DNN models implemented on edge devices, the author proposed a DNN fingerprinting technique in Wang et al. [97]. For the image classification task, the first-pixel value in the image was scaled to [0,1] for simplicity; intrinsic examples were generated using cross-entropy loss for the DNN model and projected gradient loss. Several random perturbations were added to improve the robustness of generated intrinsic examples. Finally, the Min-Max robust optimisation was applied to the intrinsic examples to incorporate weight perturbations from an optimisation perspective. Experiments were conducted using CIFAR-10, SVHN and ImageNet datasets on CNN and VGG-16 models. The model regards all pruning as compression attacks with an intrinsic score of 100 %. The author proposed a robust neural fingerprinting method that fingerprints the inner decision area of the model [98]. The approach works in two stages: shadow models are trained from scratch during the meta-data generation stage, and negative and positive models are generated by applying data augmentation to the shadow models. Images are optimised by applying triple loss to them during the meta-training stage, and only the inner-class distance of positive classes is reduced. The verification algorithm predicts whether the model is a copy of the original model. The experiment results showed that the accuracy of the original task remains unchanged, and the approach is robust and immune to parameter pruning and model fine-tuning attacks. For IP protection, the author proposed a task-agnostic model fingerprinting framework [99]. The approach uses obfuscation techniques to generate a set of negative and positive suspect models from the original model. The adaptive fingerprint and meta-verifier are optimised together to generate an adaptive fingerprint pair (X, V) where X represents the fingerprint and V represents the meta-verifier. The meta-verifier compares the suspect model's matching rate to the original model to determine if it is a stolen copy. The framework was evaluated on multiple network architectures using three standard datasets. Results showed that the framework has high true positive and true negative rates; even though it takes more time to construct the fingerprint, the verification process takes less time when compared with other approaches. The author proposed a testing-based approach to detect model reuse by comparing the similarity of the deep learning model [100]. The approach uses a decision distance vector (DDV) to represent behavioural patterns, and similarity is measured using cosine similarity. First, the original and suspect models are chosen, and the DDV of both models is calculated. The similarity between the two models is calculated using cosine similarity between the DDVs of the two models. A threshold value is computed to determine if the suspect model is a copied model. The results were evaluated on ResNet18 and MobileNetV2 models using three standard datasets and showed a high correctness rate. The method can effectively detect copied models even after transfer learning, model compression and model stealing attacks. To identify model integrity breaches with high accuracy, the author proposed a sensitive-sample fingerprinting approach [101]. A small set of tweaked inputs known as sensitive samples were generated by introducing small perturbations to the normal inputs and using them as fingerprints. To verify the model's integrity, the user can generate output from the suspect model using sensitive samples and transmit the result to the cloud provider, who compares the original model output with the output

Table 8

Comparison of watermarking, fingerprinting and encryption techniques.

|                             | Affects Accuracy | Non-Invasive | Privacy Preserving | Model Integrity | Easy to implement | Effective | Efficient | Ownership verification |
|-----------------------------|------------------|--------------|--------------------|-----------------|-------------------|-----------|-----------|------------------------|
| Non-reversible watermarking | ✓                | ×            | ×                  | ×               | ✓                 | ✓         | ✓         | ✓                      |
| Reversible watermarking     | ✓                | ×            | ×                  | ✓               | ×                 | ✓         | ✓         | ✓                      |
| Fingerprinting              | ×                | ✓            | ×                  | ✓               | ×                 | ✓         | ×         | ✓                      |
| Encryption                  | ✓                | ×            | ✓                  | ×               | ×                 | ✓         | ×         | ×                      |

submitted by the user to check if the model has been modified or is the same as earlier. To generate sensitive samples a sample selection algorithm called the maximum active neuron cover was also proposed. Experiment results indicated that model integrity breaches can be detected with high accuracy and zero false positives. The approach is immune to model fine-tuning and parameter-pruning attacks. The author proposed a task-agnostic fingerprinting algorithm for DNN fingerprinting [102]. The approach is independent of training data and may be applied to various downstream tasks. To generate fingerprints, inherent properties of the target DNN model are used, specifically, the linear regions formed by the activation function ReLU. Given a random seed, a small perturbation is identified, which preserves the activation pattern in the given layer. The generated fingerprints are made up of an n-pair of samples with the same activation pattern at the first ReLU layer. In both the white-box and black-box settings, the verification algorithm checks whether the constructed samples are in the same linear region as the target model. Three standard datasets were used to evaluate the approach on ResNet-18 and FCN models. The results indicated that the approach is immune to fine-tuning, retraining and parameter pruning attacks and has high ARUC values.

From the above literature, we can conclude the following:

- Fingerprinting provides a non-invasive way to protect the IP of the model owner; fingerprints are non-invasive and do not change the model parameters in any manner, maintaining the model's integrity.
- Fingerprinting works by extracting data points near the classification boundary of the original model and using them as fingerprints.
- Finding fingerprints is an extensive task and requires more time than other approaches.
- Fingerprints are not secure against attacks that change the classification boundaries of the original model.

These DNN fingerprinting technique are summarized in Table 6.

### 2.3. Model security using encryption

The above two approaches, watermarking and fingerprinting, focus on protecting the model owner's IP. To do so, the verification step determines if the model is original, or a replica based on the visibility of the owner's watermark or fingerprint. Adversaries can still use various attacks like side-channel [103] or model reversal attacks to extract the parameters from the original model. Encryption offers the solution to protect the model against IP theft and maintain privacy [104]. Encrypting all the model parameters will result in a heavy computation burden on the original task, so the model encryption operates by selecting a few critical parameters that contribute to the classification task accuracy. These critical parameters are encrypted using a secret key generated by the model owner. Even if an adversary accesses the encrypted model or replicates it by copying the parameters from the original model, the classification accuracy is inferior without the secret key, rendering it worthless. The secret key is distributed to the authorised user along with the encrypted model. The authorised users can decrypt these important parameters using the secret key and use the model for their tasks. A general approach to model encryption is depicted in Fig. 12.

An active approach to protecting DNN IP by encrypting the weight parameters of the DNN model was proposed by the author in Xue et al. [104]. The approach works in three steps: first, a few labelled images are selected from the training dataset, known as the encryption set. Next, the model owner randomly selects some convolution and fully connected layers from the model. Using an encryption set, the adversarial perturbation is iteratively added to the weights by perturbing only one weight of one selected hidden layer in each iteration. Finally, the encryption set is fed into the model to compute the value of the loss function. The secret key comprises the pair, position of weight and perturbation. To decrypt, the user must use the secret key, remove the perturbation and obtain the original model. Experiments were conducted on DenseNet, ResNet and AlexNet models using standard datasets. The results revealed a slight degradation in accuracy and immunity to model fine-tuning, model pruning and adaptive attacks, but verifying the model ownership is impossible. The author proposed a selective encryption algorithm to protect neural network models from unauthorised access and access control [105]. The approach works by feeding a pre-trained model into the select module, which selects the important parameters to encrypt. The encryption module encrypts the important parameters and constructs the encrypted model. To decrypt the model, the user requires permission. The user would feed the encrypted model and permission into the decryption module to get the original model. Permission is used for decryption and access control. Experiments were conducted on the VGG19 and DCNN models using CIFAR10 and ImageNet datasets, respectively. The results indicated that the model performs differently for various permissions and requires only 8 % encrypted parameters to succeed. The approach is immune to denoising and retraining attacks. A secure multi-party computation framework for privacy preservation was proposed by the author [106]. The approach works in two phases: in the initialisation phase, first, a fully homomorphic encryption-compatible DNN is designed and trained. A set of essential neurons is selected and encrypted using homomorphic encryption. In the evaluation phase, the user evaluates testing samples. The unencrypted neuron evaluates plaintext. The encrypted essential neurons evaluate the ciphertext, and the output of the encrypted neuron is obfuscated and sent to the model owner for decryption; after that, it is sent back to the user, where it is de-obfuscated. The output from plaintext neurons and decrypted output is used to evaluate the testing samples. Experiments were performed on multiple network architectures using standard datasets. The results showed that the approach protects both model and data privacy. It is observed that there is a performance-security trade-off, where a smaller number of encrypted essential neurons results in saving computation but provides weaker protection. The author presented a novel way to protect against DL IP theft in neural processing units (NPUs) using a keyed model obfuscation scheme [107]. Obfuscation relies on the basic structure of DNN and can be accomplished using three possible approaches: row/column swap, in which rows and columns from the same filter within the same layer are randomly selected and swapped; full filter swap, in which the entire filter is swapped; and hybrid approach, in which filter, and

row/column swaps are mixed. The authorised user requires a mapping control key to generate the original model. The experiment results showed that model accuracy degrades dramatically for unauthorised users, preventing information leakage. In [103], the author proposed a novel framework based on a chaotic map to protect the IP of DNNs. Arnold's cat map, a chaotic map, is used to encrypt and decrypt the weights of the DNN model. The positions of convolutional and fully connected kernels of the DNN are exchanged using Arnold's cat map. The secret key is obtained during the encryption phase, where the optimal encrypted model with the lowest decryption time is selected with its corresponding secret key. The model can be decrypted using the secret key on encrypted convolution kernels using the inverse Arnold's cat map. The experiment results indicated low accuracy loss and a slight impact on inference speed. To protect the DNN model and classify images while preserving privacy, the author proposed an encryption method [108]. The method works by encrypting both the model and test images. Image transformation has three steps: first, images are partitioned into blocks; pixel shuffling, bit flipping, and normalisation are applied on each block to transform them; and finally, blocks are integrated to get a modified test image. The model is encrypted by flattening the patches, mapping them to vectors and then performing pixel shuffling and bit flipping with the same keys used in the image encryption phase. The method was evaluated on ConvMixer using the CIFAR-10 dataset. The results showed that the model is robust against random key attacks but is computationally intensive. In [109], the author proposed a model protection method using block-wise pixel shuffling. A secret key was used to preprocess the images before the training or testing. To preprocess the images, block-wise pixel shuffling was performed using the secret key. Experiments were conducted on the ResNet18 model using the CIFAR-10 dataset. The results indicated that the method has accuracy similar to traditional watermarking methods and is robust against fine-tuning attacks; however, it needs a way to secure the key.

Observing the above literature, the following can be concluded:

- Encrypting the DNN model provides a way to ensure its privacy. Weight parameters are encrypted so that even though the attackers acquire the complete model, they can still not extract and replicate the model without sacrificing significant accuracy levels.
- DNN model parameters are selected based on the importance of the parameters for the classification task and encrypting them with a secret key.
- Encrypting the DNN model requires immense computational power and substantial computational time, and in most cases, the ownership of the model cannot be verified.

The above literature and observations are summarized in Table 7.

### 3. Challenges

Watermarking, fingerprinting and encryption provides a way to secure different aspects of the Watermarking, fingerprinting and encryption are techniques to secure different aspects of the DNN model, as depicted in Fig. 13. After investigating various watermarking, fingerprinting and encryption approaches, we present below the challenges that we have encountered.

#### • White-box vs Black-box watermarking

The white-box approach is comparatively easy to implement and consumes less time than the black-box approach; however, it can only be implemented if the user has complete access to the model. Nowadays, most of the DNN models are hosted by owners on their own servers, and complete access to them is not possible; the black-box approach proves useful in this scenario. It lets a user verify the ownership in a black-box manner, but is time-consuming. A combined technique that provides black-box ownership verification while having trust and simplicity of the white-box approach can provide a possible solution.

#### • Need for an efficient approach

Encryption preserves the privacy of the DNN model, whereas fingerprints and watermarks are used to verify ownership and integrity authentication. While watermarks are easy to generate, they require considerable time and computation power to embed and verify the watermark. In contrast, fingerprints require precise calculations to find perturbations for extraction of fingerprints and are easily malleable if the model's classification boundary changes. Encryption encrypts the model parameters to maintain the privacy of the DNN model. Encryption and decryption incur extra computation burdens for the DNN model. An efficient approach that minimises embedding/encryption, verification/decryption time and computation cost while protecting the model ownership is required.

#### • Intrusive vs non-intrusive approach

Both fingerprints and watermarks can be used to protect the IP of the model's owner. While watermarking is an intrusive approach that negatively impacts the classification accuracy of the original model, fingerprinting is a non-intrusive method to protect the same; however, it is rendered useless if the model's classification boundary is changed by an attacker. A watermarking technique that is non-intrusive and provides the same level of IP protection can be a likely solution.

#### • Focus only on classification task

Most of the encryption, fingerprinting and watermarking techniques available are designed to work only for classification tasks. A generalised technique that can be used for all the tasks (text processing, image recognition, classification, etc.) can be considered.

#### • Privacy preservation vs ownership verification

While fingerprinting and watermarking can be used to verify the ownership of the DNN model and integrity authentication, they cannot be used to preserve the privacy of the model; the attackers can still access the weight parameters of the model to construct their own model for their personal use. Encryption is used to preserve the privacy of the DNN model but cannot be used for integrity

authentication or verification of the ownership of the model. A watermarking technique that encrypts the model's parameters in the embedding phase while verifying the ownership and integrity of the model can be a feasible potential solution.

#### • Security against adaptive attacks and hybrid attacks

Most of the watermarking and fingerprinting schemes available are not secured against adaptive and hybrid attacks where the user uses multiple attacks to erase or collude the embedded watermark and fingerprints. A secure watermarking scheme that embeds watermarks dynamically to evade adaptive and hybrid attacks can be a possible solution.

- Optimising the generation and verification process for efficient watermarking and fingerprinting algorithms is an ongoing challenge.

Based on the above study, it is a complex task to compare and conclude which approach (fingerprinting, watermarking or encryption) is better for a particular application. We have attempted to compare the three approaches and summarised our findings in Table 8. It is observed that watermarking is a more effective technique for model ownership verification. However, fingerprinting and encryption are critical for preserving model integrity and privacy, respectively. Therefore, a combination of watermarking, fingerprinting and encryption-based model security can be proposed to make the model more robust and secure for practical applications. Further, time cost and computational cost should be minimum for any real time applications.

## 4. Conclusion

In this survey, we have systematically studied many prominent techniques for model security in watermarking, fingerprinting and encryption domains over recent years. As copyright violation of DL models becomes an emerging threat and a growing concern to our research community, model security has become a popular research subject in various applications. We have discussed and summarised the recent state-of-the-art security solutions for DL models in watermarking, fingerprinting and encryption domains. Our study explores the recent applications, possible attacks, current limitations and notable suggestions. Further, this comprehensive survey explores the recent research gaps and opportunities in technical detail; thereby, we hope to empower researchers and practitioners to provide more secure solutions for DL models. Moreover, the extensive survey is the first to consider model security through different notable techniques together, simultaneously. Despite its significant improvements, the existing methods still has limitations and presents opportunities for further development. Its current focus on model security restricts its application scope, and future research could expand its use to media security. Enhancing model and media security strategies at the same time is a promising area of exploration.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgement

All authors approved the version of the manuscript to be published.

## Data availability

No data was used for the research described in the article.

## References

- [1] Dimiduk DM, Holm EA, Niezgoda SR. Perspectives on the impact of machine learning, deep learning, and artificial intelligence on materials, processes, and structures engineering. *Integr Mater Manuf Innov* 2018;7:157–72.
- [2] Muniasamy A, Alasiry A. Deep learning: the impact on future eLearning. *Int J Emerg Technol Learn (Online)* 2020;15(1):188.
- [3] Abd Elaziz M, Dahou A, Abualigah L, Yu L, Alshinwan M, Khasawneh AM, Lu S. Advanced metaheuristic optimization techniques in applications of deep neural networks: a review. *Neural Comput Appl* 2021;33:1–21.
- [4] Chen J, Lin S. An interactive neural network-based approach for solving multiple criteria decision-making problems. *Decis Support Syst* 2003;36(2):137–46.
- [5] Deng L, Yu D. Deep learning: methods and applications. *Found Trends® Signal Process* 2014;7(3–4):197–387.
- [6] Khan S, Yairi T. A review on the application of deep learning in system health management. *Mech Syst Signal Process* 2018;107:241–65.
- [7] Mamoshina P, Vieira A, Putin E, Zhavoronkov A. Applications of deep learning in biomedicine. *Mol Pharm* 2016;13(5):1445–54.
- [8] Mahdavi S, Ghorbani AA. Application of deep learning to cybersecurity: a survey. *Neurocomputing* 2019;347:149–76.
- [9] Shankar, D., Narumanchi, S., Ananya, H.A., Kompalli, P., & Chaudhury, K. (2017). Deep learning based large scale visual recommendation and search for e-commerce. *arXiv preprint arXiv:1703.02344*.
- [10] Zhang Z, He Q, Gao J, Ni M. A deep learning approach for detecting traffic accidents from social media data. *Transp Res Part C* 2018;86:580–96.
- [11] Chong E, Han C, Park FC. Deep learning networks for stock market analysis and prediction: methodology, data representations, and case studies. *Expert Syst Appl* 2017;83:187–205.
- [12] Lampe B, Meng W. A survey of deep learning-based intrusion detection in automotive applications. *Expert Syst Appl* 2023;221:1–23.
- [13] Gee JP. Deep learning properties of good digital games: how far can they go?. *Serious games*. Routledge; 2009. p. 89–104.
- [14] Deng L, Hinton G, Kingsbury B. New types of deep neural network learning for speech recognition and related applications: an overview. In: *2013 IEEE international conference on acoustics, speech and signal processing*. IEEE; 2013. p. 8599–603.
- [15] Goel A, Goel AK, Kumar A. The role of artificial neural network and machine learning in utilizing spatial information. *Spat Inf Res* 2023;31(3):275–85.
- [16] Archana R, Jeevaraj PE. Deep learning models for digital image processing: a review. *Artif Intell Rev* 2024;57(1):11.



- [17] Hemanth DJ, Estrela VV, editors. Deep learning for image processing applications (Vol. 31). IOS Press; 2017.
- [18] AI image statistics: how much content was created by ai." <https://journal.everyapixel.com/ai-image-statistics>, 2023. Accessed Dec. 18, 2023.
- [19] S. Kemp, "Digital 2023: global overview report." DataReportal –Global Digital Insights, 2023. Accessed: Dec. 18, 2023.
- [20] Li Y, Wang H, Barni M. A survey of deep neural network watermarking techniques. *Neurocomputing* 2021;461:171–93.
- [21] Hsieh IC, Li CT. Netfense: adversarial defenses against privacy attacks on neural networks for graph data. *IEEE Trans Knowl Data Eng* 2021;35(1):796–809.
- [22] <https://dl.acm.org/doi/fullHtml/10.1145/3381831>.
- [23] <https://www.ri.se/en/news/blog/generative-ai-does-not-run-on-thin-air>.
- [24] Uchida Y, Nagai Y, Sakazawa S, Satoh SI. Embedding watermarks into deep neural networks. In: *Proceedings of the 2017 ACM on international conference on multimedia retrieval*; 2017. p. 269–77.
- [25] Singh AK, Dave M, Mohan A. A novel technique for digital image watermarking in frequency domain. In: *2012 2nd IEEE international conference on parallel, distributed and grid computing*; 2012. p. 424–9. <https://doi.org/10.1109/PDGC.2012.6449858>.
- [26] Singh OP, Singh AK, Srivastava G, Kumar N. Image watermarking using soft computing techniques: a comprehensive survey. *Multimed Tools Appl* 2021;80(20):30367–98.
- [27] Adi Y, Baum C, Cisse M, Pinkas B, Keshet J. Turning your weakness into a strength: watermarking deep neural networks by backdoor. In: *27th USENIX security symposium (USENIX security 18)*; 2018. p. 1615–31.
- [28] Lin CY, Chang SF. Watermarking capacity of digital images based on domain-specific masking effects. In: *Proceedings international conference on information technology: coding and computing*. IEEE; 2001. p. 90–4.
- [29] Chu SC, Huang HC, Shi Y, Wu SY, Shieh CS. Genetic watermarking for zerotree-based applications. *Circuits, Syst Signal Process* 2008;27:171–82.
- [30] Boenisch F. A systematic review on model watermarking for neural networks. *Front Big Data* 2021;4:729663.
- [31] Regazzoni F, Palmieri P, Smalbegovic F, Cammarota R, Polian I. Protecting artificial intelligence IPs: a survey of watermarking and fingerprinting for machine learning. *CAAI Trans Intell Technol* 2021;6(2):180–91.
- [32] Fkirin A, Attiya G, El-Sayed A, Shouman MA. Copyright protection of deep neural network models using digital watermarking: a comparative study. *Multimed Tools Appl* 2022;81(11):15961–75.
- [33] Sun, Y., Liu, T., Hu, P., Liao, Q., Fu, S., Yu, N., Guo, D., Liu, Y., Liu, L. (2023). Deep intellectual property: a survey. *arXiv preprint arXiv:2304.14613*.
- [34] Lee S, Song W, Jana S, Cha M, Son S. Evaluating the robustness of trigger set-based watermarks embedded in deep neural networks. *IEEE Trans Dependable Secure Comput* 2023;20(4):3434–48.
- [35] Xue M, Zhang Y, Wang J, Liu W. Intellectual property protection for deep learning models: taxonomy, methods, attacks, and evaluations. *IEEE Trans Artif Intell* 2021;3(6):908–23.
- [36] Naresh VS, Thamarai M, Allavarpu VD. Privacy-preserving deep learning in medical informatics: applications, challenges, and solutions. *Artif Intell Rev* 2023;56(Suppl 1):1199–241.
- [37] Liu X, Xie L, Wang Y, Zou J, Xiong J, Ying Z, Vasilakos AV. Privacy and security issues in deep learning: a survey. *IEEE Access* 2020;9:4566–93.
- [38] Zhang Q, Xin C, Wu H. Privacy-preserving deep learning based on multiparty secure computation: a survey. *IEEE Internet Things J* 2021;8(13):10412–29.
- [39] Lukas N, Jiang E, Li X, Kerschbaum F. Sok: how robust is image classification deep neural network watermarking?. In: *2022 IEEE symposium on security and privacy (SP)*. IEEE; 2022. p. 787–804.
- [40] Peng S, Chen Y, Xu J, Chen Z, Wang C, Jia X. Intellectual property protection of DNN models. *World Wide Web* 2023;26(4):1877–911.
- [41] Xu G, Li H, Ren H, Yang K, Deng RH. Data security issues in deep learning: attacks, countermeasures, and opportunities. *IEEE Commun Mag* 2019;57(11):116–22.
- [42] Jia H, Choquette-Choo CA, Chandrasekaran V, Papernot N. Entangled watermarks as a defense against model extraction. In: *30th USENIX security symposium (USENIX security 21)*; 2021. p. 1937–54.
- [43] Fredrikson M, Jha S, Ristenpart T. Model inversion attacks that exploit confidence information and basic countermeasures. In: *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*; 2015. p. 1322–33.
- [44] Chang S, Li C. Privacy in neural network learning: threats and countermeasures. *IEEE Netw* 2018;32(4):61–7.
- [45] Abadi M, Chu A, Goodfellow I, McMahan HB, Mironov I, Talwar K, Zhang L. Deep learning with differential privacy. In: *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*; 2016. p. 308–18.
- [46] Orlandi C, Piva A, Barni M. Oblivious neural network computing via homomorphic encryption. *EURASIP J Inf Secur* 2007;2007:1–11.
- [47] Wagh S, Gupta D, Chandran N. SecureNN: 3-party secure computation for neural network training. In: *Proceedings on privacy enhancing technologies*; 2019.
- [48] Tramer, F., & Boneh, D. (2018). Slalom: fast, verifiable and private execution of neural networks in trusted hardware. *arXiv preprint arXiv:1806.03287*.
- [49] Guan X, Feng H, Zhang W, Zhou H, Zhang J, Yu N. Reversible watermarking in deep convolutional neural networks for integrity authentication. In: *Proceedings of the 28th ACM international conference on multimedia*; 2020. p. 2273–80.
- [50] Xu, X., Li, Y., & Yuan, C. (2019). A novel method for identifying the deep neural network model with the serial number. *arXiv preprint arXiv:1911.08053*.
- [51] Fan L, Ng KW, Chan CS. Rethinking deep neural network ownership verification: embedding passports to defeat ambiguity attacks. *Advances in neural information processing systems*, 32; 2019.
- [52] Guo J, Potkonjak M. Watermarking deep neural networks for embedded systems. In: *2018 IEEE/ACM international conference on computer-aided design (ICCAD)*. IEEE; 2018. p. 1–8.
- [53] Chen X, Wang W, Bender C, Ding Y, Jia R, Li B, Song D. Refit: a unified watermark removal framework for deep learning systems with limited data. In: *Proceedings of the 2021 ACM Asia conference on computer and communications security*; 2021. p. 321–35.
- [54] Zhang J, Gu Z, Jiang J, Wu H, Stoecklin MP, Huang H, Molloy I. Protecting intellectual property of deep neural networks with watermarking. In: *Proceedings of the 2018 on Asia conference on computer and communications security*; 2018. p. 159–72.
- [55] Molchanov, P., Tyree, S., Karras, T., Aila, T., & Kautz, J. (2016). Pruning convolutional neural networks for resource efficient inference. *arXiv preprint arXiv:1611.06440*.
- [56] Yang, Z., Dang, H., & Chang, E.C. (2019). Effectiveness of distillation attack and countermeasure on neural network watermarking. *arXiv preprint arXiv:1906.06046*.
- [57] Wang B, Yao Y, Shan S, Li H, Viswanath B, Zheng H, Zhao BY. Neural cleanse: identifying and mitigating backdoor attacks in neural networks. In: *2019 IEEE symposium on security and privacy (SP)*. IEEE; 2019. p. 707–23.
- [58] Wang T, Kerschbaum F. Attacks on digital watermarks for deep neural networks. In: *ICASSP 2019-2019 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE; 2019. p. 2622–6.
- [59] Navidpour SM, Uysal M, Kavehrad M. BER performance of free-space optical transmission with spatial diversity. *IEEE Trans Wirel Commun* 2007;6(8):2813–9.
- [60] Chai T, Draxler RR. Root mean square error (RMSE) or mean absolute error (MAE). *Geosci Model Dev Discuss* 2014;7(1):1525–34.
- [61] Li Y, Tondi B, Barni M. Spread-transform dither modulation watermarking of deep neural network. *J Inf Secur Appl* 2021;63:103004.
- [62] Kuribayashi M, Yasui T, Malik A. White box watermarking for convolution layers in fine-tuning model using the constant weight code. *J Imaging* 2023;9(6):117.
- [63] Pagnotta, G., Hitaj, D., Hitaj, B., Perez-Cruz, F., & Mancini, L.V. (2022). Tattooed: a robust deep neural network watermarking scheme based on spread-spectrum channel coding. *arXiv preprint arXiv:2202.06091*.
- [64] Chen, H., Rohani, B.D., & Koushanfar, F. (2018). Deepmarks: a digital fingerprinting framework for deep neural networks. *arXiv preprint arXiv:1804.03648*.
- [65] Rouhani, B.D., Chen, H., & Koushanfar, F. (2018). Deepsigns: a generic watermarking framework for IP protection of deep learning models. *arXiv preprint arXiv:1804.00750*.
- [66] Nagai Y, Uchida Y, Sakazawa S, Satoh SI. Digital watermarking for deep neural networks. *Int J Multimed Inf Retr* 2018;7:3–16.
- [67] Kuribayashi M, Tanaka T, Funabiki N. Deepwatermark: embedding watermark into DNN model. In: *2020 Asia-Pacific signal and information processing association annual summit and conference (APSIPA ASC)*. IEEE; 2020. p. 1340–6.



- [68] Zhang J, Chen D, Liao J, Zhang W, Hua G, Yu N. Passport-aware normalization for deep model protection. *Adv Neural Inf Process Syst* 2020;33:22619–28.
- [69] Qin, J., Yang, F., Deng, J., & Lyu, S. (2023). Reversible deep neural network watermarking: matching the floating-point weights. *arXiv preprint arXiv:2305.17879*.
- [70] Lv P, Li P, Zhang S, Chen K, Liang R, Ma H, Zhao Y, Li Y. A robustness-assured white-box watermark in neural networks. *IEEE Trans Dependable Secure Comput* 2023;20(6):5214–29.
- [71] Fei J, Xia Z, Tondi B, Barni M. Supervised GAN watermarking for intellectual property protection. In: 2022 IEEE international workshop on information forensics and security (WIFS). IEEE; 2022. p. 1–6.
- [72] Yu N, Skripniuk V, Abdelnabi S, Fritz M. Artificial fingerprinting for generative models: rooting deepfake attribution in training data. In: Proceedings of the IEEE/CVF international conference on computer vision; 2021. p. 14448–57.
- [73] Chen, H., Rouhani, B.D., & Koushanfar, F. Blackmarks: Blackbox multibit watermarking for deep neural networks. *arXiv* 2019. *arXiv preprint arXiv:1904.00344*.
- [74] Le Merrer E, Perez P, Trédan G. Adversarial frontier stitching for remote neural network watermarking. *Neural Comput Appl* 2020;32(13):9233–44.
- [75] Szyller S, Atli BG, Marchal S, Asokan N. Dawn: dynamic adversarial watermarking of neural networks. In: Proceedings of the 29th ACM international conference on multimedia; 2021. p. 4417–25.
- [76] Zhu R, Zhang X, Shi M, Tang Z. Secure neural network watermarking protocol against forging attack. *EURASIP J Image Video Process* 2020;2020:1–12.
- [77] Tekgul BG, Xia Y, Marchal S, Asokan N. Waffle: watermarking in federated learning. In: 2021 40th international symposium on reliable distributed systems (SRDS). IEEE; 2021. p. 310–20.
- [78] Zhang J, Gu Z, Jang J, Wu H, Stoecklin MP, Huang H, Molloy I. Protecting intellectual property of deep neural networks with watermarking. In: Proceedings of the 2018 on Asia conference on computer and communications security; 2018. p. 159–72.
- [79] Guo J, Potkonjak M. Watermarking deep neural networks for embedded systems. In: 2018 IEEE/ACM international conference on computer-aided design (ICCAD). IEEE; 2018. p. 1–8.
- [80] Li, H., Willson, E., Zheng, H., & Zhao, B.Y. (2019). Persistent and unforgeable watermarks for deep neural networks. *arXiv preprint arXiv:1910.01226*, 2.
- [81] Li, H., Wenger, E., Shan, S., Zhao, B.Y., & Zheng, H. (2019). Piracy resistant watermarks for deep neural networks. *arXiv preprint arXiv:1910.01226*.
- [82] Wang L, Song Y, Xia D. Deep neural network watermarking based on a reversible image hiding network. *Pattern Anal Appl* 2023;26(3):861–74.
- [83] Ye Z, Zhang X, Feng G. Deep neural networks watermark via universal deep hiding and metric learning. *Neural Comput Appl* 2024;36(13):7421–38.
- [84] Wen, X., Li, Y., Jiang, W., & Xu, Q. (2023). On function-coupled watermarks for deep neural networks. *arXiv preprint arXiv:2302.10296*.
- [85] Li, Y., Maharana, S.K., & Guo, Y. (2024). Not just change the labels, learn the features: watermarking deep neural networks with multi-view data. *arXiv preprint arXiv:2403.10663*.
- [86] Hua G, Teoh ABJ, Xiang Y, Jiang H. Unambiguous and high-fidelity backdoor watermarking for deep neural networks. *IEEE Trans Neural Netw Learn Syst* 2024;35(8):11204–17.
- [87] Li L, Zhang W, Barni M. Universal BlackMarks: key-image-free blackbox multi-bit watermarking of deep neural networks. *IEEE Signal Process Lett* 2023;30:36–40.
- [88] Cao X, Jia J, Gong NZ. IPGuard: protecting intellectual property of deep neural networks via fingerprinting the classification boundary. In: Proceedings of the 2021 ACM Asia conference on computer and communications security; 2021. p. 14–25.
- [89] Lukas, N., Zhang, Y., & Kerschbaum, F. (2019). Deep neural network fingerprinting by conferrable adversarial examples. *arXiv preprint arXiv:1912.00888*.
- [90] Zheng Y, Wang S, Chang CH. A DNN fingerprint for non-repudiable model ownership identification and piracy detection. *IEEE Trans Inf Forensics Secur* 2022;17:2977–89.
- [91] Zhao J, Hu Q, Liu G, Ma X, Chen F, Hassan MM. AFA: adversarial fingerprinting authentication for deep neural networks. *Comput Commun* 2020;150:488–97.
- [92] Guan J, Liang J, He R. Are you stealing my model? Sample correlation for fingerprinting deep neural networks. *Adv Neural Inf Process Syst* 2022;35:36571–84.
- [93] Wang S, Wang X, Chen PY, Zhao P, Lin X. Characteristic examples: high-robustness, low-transferability fingerprinting of neural networks. In: International joint conferences on artificial intelligence organization (IJCAI); 2021.
- [94] Chen J, Wang J, Peng T, Sun Y, Cheng P, Ji S, Ma X, Li B, Song D. Copy, right? A testing framework for copyright protection of deep learning models. In: 2022 IEEE symposium on security and privacy (SP). IEEE; 2022. p. 824–41.
- [95] Peng Z, Li S, Chen G, Zhang C, Zhu H, Xue M. Fingerprinting deep neural networks globally via universal adversarial perturbations. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition; 2022. p. 13430–9.
- [96] Wang S, Chang CH. Fingerprinting deep neural networks-a deepfool approach. In: 2021 IEEE international symposium on circuits and systems (ISCAS). IEEE; 2021. p. 1–5.
- [97] Wang S, Zhao P, Wang X, Chin S, Wahl T, Fei Y, Lin X. Intrinsic examples: robust fingerprinting of deep neural networks. In: British machine vision conference (BMVC); 2021.
- [98] Yang K, Wang R, Wang L. MetaFinger: fingerprinting the deep neural networks with meta-training. In: IJCAI; 2022. p. 776–82.
- [99] Pan X, Yan Y, Zhang M, Yang M. Metav: a meta-verifier approach to task-agnostic model fingerprinting. In: Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining; 2022. p. 1327–36.
- [100] Li Y, Zhang Z, Liu B, Yang Z, Liu Y. ModelDiff: testing-based DNN similarity comparison for model reuse detection. In: Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis; 2021. p. 139–51.
- [101] He Z, Zhang T, Lee R. Sensitive-sample fingerprinting of deep neural networks. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition; 2019. p. 4729–37.
- [102] Pan X, Zhang M, Lu Y, Yang M. Tafa: a task-agnostic fingerprinting algorithm for neural networks. In: Computer security—ESORICS 2021: 26th European symposium on research in computer security, Darmstadt, Germany, October 4–8, 2021, proceedings, Part I 26. Springer International Publishing; 2021. p. 542–62.
- [103] Lin N, Chen X, Lu H, Li X. Chaotic weights: a novel approach to protect intellectual property of deep neural networks. *IEEE Trans Computer-Aided Des Integr Circuits Syst* 2020;40(7):1327–39.
- [104] Xue M, Wu Z, Zhang Y, Wang J, Liu W. AdvParams: an active DNN intellectual property protection technique via adversarial perturbation based parameter encryption. *IEEE Trans Emerg Top Comput* 2023;11(3):664–78.
- [105] Tian J, Zhou J, Duan J. Probabilistic selective encryption of convolutional neural networks for hierarchical services. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition; 2021. p. 2205–14.
- [106] Ren L, Liu Z, Li F, Liang K, Li Z, Luo B. PrivDNN: a secure multi-party computation framework for deep learning using partial DNN encryption. *Proc Privacy Enhanc Technol* 2024;3:1–18.
- [107] Goldstein BF, Patil VC, Ferreira VC, Nery AS, França FM, Kundu S. Preventing DNN model IP theft via hardware obfuscation. *IEEE J Emerg Sel Top Circuits Syst* 2021;11(2):267–77.
- [108] Iijima, R., & Kiya, H. (2022). An encryption method of convmixer models without performance degradation. *arXiv preprint arXiv:2207.11939*.
- [109] Pyone A, Maung M, Kiya H. Training DNN model with secret key for model protection. In: 2020 IEEE 9th global conference on consumer electronics (GCCE). IEEE; 2020. p. 818–21.